Design Partner MVP — Build Specification

Oct 22, 2026 · @Bradly Kerner

This spec turns Section 6 of the product strategy into buildable scope. Section 6 governs: where any later strategy section describes more, this spec builds Section 6 and labels the rest NOT NOW. Phase 0 is assumed passed. Status: frozen as v1.1 on October 22, 2026; any change after this needs a written revision note.

Governing decisions

These are the engineering calls this spec makes. Every later section follows them; changing one means revising this table first.

#	Decision	Why
D1	One Next.js (App Router, TypeScript) app for shop UI, public pages and internal admin; one worker process; one PostgreSQL database	Smallest system that can be secured and debugged by a small team
D2	Managed PostgreSQL (Supabase project) used as plain Postgres. Supabase Auth for login (email magic link, SMS code). Supabase Storage for photos and PDFs. All data access happens server-side through Drizzle; the browser never queries the database	Fewer vendors; RLS-capable Postgres; avoids client-side data access as an attack surface
D3	Tenant = Business. Every tenant table has `business_id`, a `UNIQUE (business_id, id)` key, composite foreign keys, and RLS keyed on a per-transaction setting	Four independent layers; the database rejects cross-tenant links even if code is wrong
D4	One `locations` row per business, created at signup, referenced by estimates, invoices, appointments and the phone number. No location UI	Cheap now; avoids a painful backfill when multi-location arrives

#	Decision	Why
D5	No separate Contact table. Customer holds one mobile phone and one email. Conversations and consent are keyed by phone number, not customer	Two customers can share a phone; routing and STOP must follow the number
D6	Estimates are versioned: `estimate` (header) → `estimate_version` (frozen at send) → `estimate_line`. Customer decisions live on the `authorization`, not on lines	Lines never mutate after send; authorization points at the exact version
D7	Invoices are not versioned. A sent invoice is immutable; to change it, void it and issue a replacement that references it	Standard accounting practice; simpler than invoice versions
D8	One payment pays one invoice (`payment.invoice_id`). No allocation table in MVP; refunds mirrored from Stripe as `payment_refund` rows	No partial-payment UI exists; an allocation table can be introduced by migration later
D9	Stripe Connect with Stripe-managed pricing: Standard connected accounts (shop has its own full Stripe dashboard), direct charges, Stripe Checkout Sessions created on demand from the public invoice page	No platform fees, no card data, refunds and disputes handled in the shop's dashboard
D10	Twilio ISV architecture #1: one subaccount and one Messaging Service per shop; Secondary Customer Profile, Brand and Campaign per shop	Twilio's recommended isolation; one bad shop cannot suspend others
D11	Imported sales become `historical_sale` records, never fake invoices or payments	Financial documents in our system are only ones we actually issued
D12	Service due is owner-entered only (a date and description on the vehicle). No mileage projection	Section 6 forbids fabricated predictions

#	Decision	Why
D13	Recovered revenue is measured on paid invoice subtotal before tax, net of refunds. Direct and influenced are separate records with stored evidence	Revenue the shop keeps; explainable line by line
D14	Legal copy (authorization text, SMS consent language, retention periods) lives in versioned, attorney-approved `legal_policy` records per jurisdiction. Production refuses to render a document whose policy is unapproved	No hard-coded legal text; no invented law
D15	Owner and Staff share all day-to-day permissions. Three account-level actions are Owner-only: managing the staff login, Stripe onboarding, texting registration submission	Pushback on "identical permissions": these actions bind the business legally and financially; the role column already exists, no role UI is built
D16	Recovery outreach requires an outreach-eligible consent basis per phone number, defined with counsel (Open Question 1). Until then, the system defaults to the conservative rule in Section 14	The single biggest legal risk in the product

Revision 1.1 (October 22, 2026), applied throughout this spec:

1. **Invoices can never exceed the authorization.** An invoice contains only approved lines at authorized quantities and prices; the shop may only reduce. Added or increased work requires a supplement: revise the authorized estimate, and the customer re-decides every line (F13b).
2. **No preselected decisions.** Every estimate line starts undecided; Submit stays disabled until the customer has chosen Approve or Decline on each line.
3. **Direct attribution counts only causally connected work.** Opportunity-specific outreach (declined work, open estimate, unpaid invoice) counts only the work tied to that opportunity; visit-generating outreach (lapsed, service due) counts the whole visit.
4. **Independent penetration test moves to commercial V1.** The design-partner launch requires an internal security review instead, unless an external test is quick and inexpensive.

1. System boundary

The MVP is a money recovery tool with just enough shop operations (customers, vehicles, estimates, invoices, payments, texting) to create and collect the money it finds. Ambiguities resolve toward the smaller build unless that would damage a foundation.

Area	IN SCOPE	OUT OF SCOPE (NOT NOW)
Customers	Name, mobile phone, email, notes, "do not contact" flag, archive	Fleets, companies with many contacts, households, custom fields, tags, customer addresses
Vehicles	Year/make/model/trim/engine, VIN with decode, plate, mileage readings, next-service date and description (owner-entered), mark sold/inactive	Mileage-based service prediction, vehicle photos, CARFAX/plate lookup
Notes	Plain-text notes on customer or vehicle	Attachments on notes, mentions
Timeline	One chronological feed per customer from `activity`	Filtering by type beyond a simple toggle, exports
Search	Name, phone (any format), email, plate, VIN (full or last 6), estimate number, invoice number	Note full-text search, fuzzy address search
Estimates	Labor, parts and fee lines; taxable flag per line; one tax rate per business; up to 10 photos; versions; send by text or email; public review page; per-line approve/decline; signature; authorization record; PDF	Service catalog beyond a short saved-lines list (max 25), discounts, deposits, packages, labor guide, parts pricing
Saved lines	Owner can save up to 25 common lines (description, type, price) for quick add	Categories, pricing matrix
Declined work	Recommendation record per declined line, survives estimate closure, status lifecycle, resolution link	Automatic re-quote

Area	IN SCOPE	OUT OF SCOPE (NOT NOW)
Invoices	Create from approved lines of an authorized version; while draft, only reduce (remove a line, lower a quantity or price); send; PDF; void and reissue; paid/unpaid status	Invoices without an estimate (NOT NOW; see Open Question 5), deposits, partial-payment UI, credit memos, statements
Payments	Card via Stripe Checkout; cash; check; status mirrored from Stripe; refunds and disputes mirrored read-only	ACH, card on file, refund UI, Tap to Pay, payment plans
Texting	One number per shop, two-way inbox, delivery status, STOP/START/HELP, consent ledger, registration workflow with status, email fallback	MMS sending from shop (inbound MMS shown as "photo received", stored), voice, missed-call text-back, AI replies
Email	Transactional email for estimates, invoices, receipts, reminders, login, recovery email fallback	Marketing campaigns, newsletters
Reminders	Invoice reminder, estimate follow-up; on/off; fixed timing; editable template	Any other automation, custom timing
Money Finder	Five lists, item selection, editable message, send, outreach status, dismiss and snooze	Automatic sending, AI suggestions, mileage-based service due
Attribution	Direct and influenced records with evidence, recovered revenue report, owner reversal, support manual correction	Multi-touch weighting, ROI charts beyond a monthly total
Schedule	Today and tomorrow lists; create/edit/cancel an appointment (customer, vehicle, time, note)	Calendar views, tech assignment, dispatch, online booking, appointment reminder texts
Access	Owner + one Staff login; magic link or SMS code; MFA via SMS code on new device	Role configuration, more than 2 users, SSO
Import	Concierge import by our team from Square, QuickBooks or CSV, dry run, report, reversal	Self-serve import UI, column-mapping UI, merge UI

Area	IN SCOPE	OUT OF SCOPE (NOT NOW)
Settings	Business profile, tax rate, saved lines, payments status, texting registration, reminders and templates, team, support access log	Branding beyond logo, custom domains
Our billing	Manual invoicing outside the product	Self-serve SaaS billing, plans, entitlements UI
Internal admin	Tenant list, view-as (read-only), import console, registration ops, webhook replay, legal policy manager, attribution corrections, data export run	Customer-facing support chat
Data export	Support-run full export (CSV + files) on owner request	Self-serve export button
Platform	Installable responsive web app (PWA manifest, home-screen icon)	Native apps, offline mode, push notifications

2. Personas and permissions

Four personas. Owner and Staff are the same in the shop; three account actions are Owner-only (D15). The `membership.role` column exists with values `owner` and `staff`; no role UI exists.

Capability	Owner	Staff	Customer (public link)	Internal support/admin
View Money Finder, customers, vehicles, timeline, estimates, invoices, payments, inbox, schedule	Yes	Yes	No	Read-only via view-as, time-boxed and logged
Create/edit customers, vehicles, notes, appointments	Yes	Yes	No	No

Capability	Owner	Staff	Customer (public link)	Internal support/admin
Create, edit draft, send, revise, void estimates	Yes	Yes	No	No
Create, send, void invoices; record cash/check	Yes	Yes	No	No
Send Money Finder outreach, dismiss/snooze items	Yes	Yes	No	No
Reply in inbox	Yes	Yes	Replies by SMS only	No
Reverse an attribution ("not from outreach")	Yes	Yes	No	Manual direct attribution with reason
Edit business profile, tax rate, saved lines, reminder settings, templates	Yes	Yes	No	No
Invite/remove the Staff login	**Owner only**	No	No	Can reset on owner request
Start/resume Stripe onboarding	**Owner only**	No	No	View status only
Submit texting registration (attestations)	**Owner only**	No	No	Can correct data with owner on the phone; submission action logged
Delete a customer	**Owner only** (archive for Staff)	Archive only	No	No
View support access log and audit events	Yes	No	No	Yes

Capability	Owner	Staff	Customer (public link)	Internal support/admin
View the one estimate version or invoice the link points to; approve/decline lines; sign; pay	No	No	Yes, that document only	No
Run imports, replay webhooks, manage legal policies, run data export	No	No	No	Yes (admin role), all audited

Customer (public link): an unauthenticated visitor holding a valid, unexpired, unrevoked token. Sees only the shop's name, logo, address and phone, the customer's name, the vehicle summary, and the document. Never sees other documents, history or notes.

Internal support/admin: our staff, signing in to `/admin` with Google Workspace SSO restricted to our domain plus an allowlist, MFA enforced. Two internal roles: `support` (view-as, registration help) and `admin` (imports, replay, policies, corrections). View-as is read-only, requires a written reason, lasts 60 minutes, and appears in the owner's support access log.

3. Information architecture

The app opens on Money, not on a calendar. Four tabs plus a New button; the commercial V1 "Work" board is NOT NOW because the MVP has no jobs board.

Mobile bottom bar (left to right): Money · Customers · (+ New) · Inbox · Today

- **Money** (home): Money Finder overview, recovered revenue card, unpaid and open items. Badge: none.
- **Customers**: search-first list.
- **+ New** (sheet): Customer, Estimate, Appointment, Record payment. Each starts with a customer picker except New customer.
- **Inbox**: conversations; badge = unread count.
- **Today**: today's and tomorrow's appointments.
- **Settings**: avatar menu, top right of every tab. Not a tab.
- **Search**: magnifier in the top bar of every tab, opens the global search overlay.

Desktop (≥ 1024 px): the same four destinations in a left sidebar, a "New" button at its top, search field in the top bar, settings under the avatar. Detail screens open in the main pane; no multi-pane layouts in MVP.

Screen hierarchy

- Money
  - Money Finder overview
  - Opportunity list (one per category)
  - Outreach composer → outreach batch results
  - Recovered revenue report → attribution detail
- Customers
  - Customer list and search
  - Customer detail (timeline, vehicles, estimates and invoices, notes)
    - Vehicle detail
    - Estimate editor → estimate detail (versions, authorization evidence)
    - Invoice detail → record payment
- Inbox
  - Conversation list → conversation thread
- Today
  - Today and tomorrow lists → appointment sheet
- Settings
  - Business profile and tax · Saved lines · Payments · Texting · Reminders and templates · Team · Support access log
- Onboarding (first login only): business profile → setup checklist
- Public (no login): estimate review → authorization confirmation · invoice view → payment result · link expired · email unsubscribe
- Internal `/admin`: tenants · view-as · imports · texting registrations · webhooks · legal policies · attribution corrections · data export

4. Screen inventory

43 screens: 30 shop, 6 public, 7 internal. Shared state rules apply everywhere unless a row says otherwise, so each row lists only what is specific to that screen.

Shared rules

- **Loading:** skeleton of the final layout; no spinners over 300 ms on lists. Actions show an in-button spinner and disable double-submit.
- **Error:** inline message in plain words plus Retry; form errors next to the field; the typed input is never lost. Server errors show a reference ID that support can search.
- **Permission:** a route the user may not use returns the shop's Money screen with a toast; Owner-only actions are hidden for Staff and rejected server-side.
- **Mobile:** tap targets ≥ 44 px, primary action in a sticky bottom bar, numeric keypad for money and phone, phone numbers tap-to-call and tap-to-text, camera opens directly for photos.
- **Provider-dependent actions** (send text, take card payment) show the provider state instead of failing: "Texting not active yet — sending by email."

Shop app

#	Screen	Purpose / user	Data shown	Primary actions	Secondary actions	Empty state	Specific notes
S1	Money Finder overview (home)	See all recoverable money at a glance; Owner and Staff	Five category cards: count, dollar value (or "value unknown"), oldest item age; recovered revenue this month (direct, then influenced labeled); unpaid invoices total	Open a category	Open recovered revenue report; open setup checklist if incomplete	Before import: "Your money list fills in after we import your sales history" + checklist link	Cards sorted by dollar value; service due card hidden until at least one vehicle has a next-service date
S2	Opportunity list	Pick who to contact in one category	Items: customer, vehicle, reason sentence ("Declined front brakes $612 on Mar 3"), amount, last contact, outreach status chip, eligibility warnings (no consent, opted out, cooldown)	Select/deselect items; Continue to message	Dismiss (with reason), snooze 30/90 days, open customer	"Nothing here right now" + what makes items appear	Ineligible items shown greyed with the reason, never silently hidden; select-all selects eligible only
S3	Outreach composer	Review recipients and the message, then send	Recipient count by channel (SMS, email, none), message editor with merge fields, live preview for first recipient, character/segment count, send-window notice	Send	Remove a recipient, switch template, save as shop template	n/a	Send requires a confirm step showing count and channel; disabled when 0 eligible; shows "will send at 9:00 AM" if outside the window
S4	Outreach batch results	See what happened after sending	Per recipient: delivered/failed/queued, replied, clicked, estimate created, paid, attribution	Open conversation	Retry failed by email	n/a	Live-updates every 30 s while open

#	Screen	Purpose / user	Data shown	Primary actions	Secondary actions	Empty state	Specific notes
S5	Recovered revenue report	Defensible proof of value	Month selector; direct total; influenced total in a separate block; list of attribution records with one-sentence explanation	Open attribution detail	Reverse attribution	"No recovered revenue yet this month"	Never shows a combined total
S6	Attribution detail	Explain one counted dollar amount	Evidence chain: opportunity → outreach message → interaction → estimate → invoice → payment, timestamps	Reverse (with reason)	Open any linked record	n/a	Reversed records stay visible, struck through
S7	Customer list / search	Find a customer fast	Name, phone, last visit, balance owed, open estimate flag	Open customer; New customer	Call, text from row	"No customers yet" + New customer	Search field focused on open; results as you type after 2 characters
S8	Customer detail	Everything about one customer	Header: name, phone, email, balance owed, consent status; tabs: Timeline, Vehicles, Estimates & invoices, Notes	New estimate, text, call	Edit, add vehicle, add note, new appointment, archive, mark do-not-contact; delete (Owner)	Timeline empty: "History appears here as you work"	Consent status shown as plain words ("Texts OK", "Opted out", "No text permission recorded")
S9	Customer create/edit	Add or fix a customer	Name, mobile, email, notes, consent source (if collected)	Save	Add vehicle next	n/a	Duplicate warning on phone or email match with link to existing customer
S10	Vehicle detail / edit	One vehicle's memory	Specs, VIN, plate, mileage history, next service (date + description + optional amount), estimates and invoices for this vehicle, open declined work	Edit, add mileage	Mark sold/inactive, new estimate	n/a	VIN field: paste or type; decode runs on 17 valid characters; manual entry always allowed
S11	Estimate editor	Build or revise a draft	Customer, vehicle, mileage in, lines (type, description, qty, unit price, taxable), photos, notes to customer, totals	Save draft; Send	Add saved line, reorder lines, delete draft	New draft opens with one empty line	Sticky total and Send; editing a sent estimate shows "This creates version 2" banner
S12	Estimate detail	Status and evidence	Status, version list, sent channel and time, viewed time, per-line decision, authorization evidence (signature image, name, time, IP, policy version), linked invoice	Create invoice (if authorized); Resend	Revise (new version), void, "Customer signs here" (in-shop signing), download PDF	n/a	Evidence panel read-only; superseded versions read-only

#	Screen	Purpose / user	Data shown	Primary actions	Secondary actions	Empty state	Specific notes
S13	Invoice detail	Get paid	Lines, totals, status, sent channel, payment history, refunds/disputes mirrored, reminder schedule	Send/resend; Record cash or check	Void and reissue, PDF, open in Stripe (card payments)	n/a	Draft invoices allow reductions only (remove a line, lower quantity or price), with "Add work" starting a supplement on the estimate; sent invoices read-only with "Void and reissue"
S14	Record payment sheet	Log cash or check	Amount (prefilled with balance), method, check number, date, note	Save	Cancel	n/a	Amount must equal balance (no partial UI)
S15	Inbox list	See conversations	Name or number, last message preview, unread dot, delivery failures flagged	Open thread	Filter: unread	"No messages yet"; if registration pending: registration status banner	Unknown numbers show "Not a customer yet"
S16	Conversation thread	Talk to a customer	Messages with status, linked customer(s), outreach origin tags, consent state	Send reply	Create customer, link to existing customer, new estimate, open customer	n/a	Composer disabled with reason when opted out or registration not approved
S17	Today	What's coming in	Today and tomorrow appointments: time, customer, vehicle, note, linked estimate	Open appointment	New appointment, call/text customer	"Nothing scheduled" + New appointment	Two sections only; no calendar
S18	Appointment sheet	Create/edit/cancel	Customer, vehicle, date, start time, note	Save	Cancel appointment	n/a	Date picker limited to dates; no duration math
S19	Global search overlay	Find anything	Grouped results: customers, vehicles, estimates, invoices	Open result	n/a	"No matches" + New customer with the typed text	Phone input normalized before search
S20	New sheet	Start common actions	Four actions	Choose action	n/a	n/a	One-tap from every tab
S21	Onboarding: business profile	First login	Legal name, DBA, address, phone, timezone (default America/Chicago), tax rate, logo	Continue	n/a	n/a	Owner only
S22	Setup checklist	Get to live	Steps with state: profile, payments, texting registration, import (done by us), legal policies (done by us), invite staff	Continue next step	Skip for now	n/a	Visible on Money until all steps complete

#	Screen	Purpose / user	Data shown	Primary actions	Secondary actions	Empty state	Specific notes
S23	Settings: business and tax	Maintain profile	Fields from S21, saved lines entry point	Save	n/a	n/a	Tax rate change applies to new drafts only
S24	Settings: saved lines	Quick-add lines	Up to 25 lines	Add/edit/delete	Reorder	"Save lines you use often"	Hard cap 25
S25	Settings: payments	Stripe status	Account status, charges enabled, payouts enabled, requirements due	Start/continue onboarding (Owner)	Open Stripe dashboard	Not connected: explanation + Connect	Staff sees status only
S26	Settings: texting	Registration status and number	State (six states), number once provisioned, rejection reason, required info	Continue/fix registration (Owner)	Send test text (when approved)	Not started: explanation + Start	Status copy per state defined in Section 14
S27	Settings: reminders and templates	Two reminders, templates	Toggles, fixed timing text, templates for estimate send, invoice send, reminders, five recovery categories	Toggle, edit template	Reset to default	n/a	Merge-field validation; legal footer appended and not editable
S28	Settings: team	Staff login	Owner, one Staff slot	Invite/remove Staff (Owner)	Resend invite	"Invite one teammate"	Second invite blocked with explanation
S29	Settings: support access log	Trust	View-as sessions: who, when, reason, duration	n/a	n/a	"Our team has not accessed your account"	Owner only
S30	Sign in / accept invite	Access	Email or mobile entry, code entry	Send link/code	Switch method	n/a	New device prompts SMS code (MFA)

Public pages

#	Screen	Purpose	Data shown	Primary actions	Empty/error	Notes
P1	Estimate review	Customer decides	Shop header, customer name, vehicle, photos, lines with Approve/Decline choices (none preselected), running approved total, attorney-approved authorization text, typed name, signature pad, consent checkbox for texts if offered by policy	Submit decision	Expired, revoked or superseded: explains and gives shop phone; superseded links to nothing (customer must use the new link they were sent)	Single column, works at 320 px; signature pad full-width; no login

#	Screen	Purpose	Data shown	Primary actions	Empty/error	Notes
P2	Authorization confirmation	Receipt of decision	Approved and declined lines, totals, time, link to PDF copy	Download PDF	n/a	Also emailed if email on file
P3	Invoice view	See and pay	Shop header, lines, totals, balance, status	Pay by card	Paid: "Paid on …" with receipt; void: "This invoice was replaced"	Pay button hidden when shop payments not enabled; shows shop phone instead
P4	Payment result	Return from Checkout	Success: "Payment received" (pending confirmation if webhook not yet in); cancel: back to invoice	Back to invoice	n/a	Never states "paid" from the redirect alone
P5	Link unavailable	Expired/revoked/unknown	Generic message + shop phone if token resolved, else generic only	n/a	n/a	Same response for unknown and expired tokens to avoid probing
P6	Email unsubscribe	Stop recovery emails	Confirmation	Confirm	n/a	One click; records consent event

Internal admin

#	Screen	Purpose	Primary actions	Notes
I1	Tenants	Find a shop, see setup state	Open tenant, start view-as (reason required)	Admin and support
I2	Import console	Run concierge imports	Upload, choose source profile, dry run, review report, commit, reverse	Admin only; Section 15
I3	Texting registrations	Track all shops' registration	View provider status, resubmit with corrected data, add internal note	Support and admin
I4	Webhook events	Diagnose and replay	Filter by provider/status, view payload, replay	Admin only

#	Screen	Purpose	Primary actions	Notes
I5	Legal policies	Manage attorney-approved copy	Create draft version, mark approved (records approver and document reference), retire	Admin only; approval requires second admin
I6	Attribution corrections	Fix attribution with evidence	Add manual direct attribution, reverse	Admin only; reason required
I7	Data export	Fulfill owner export requests	Generate export, deliver secure link	Admin only

5. End-to-end user flows

Each flow names the server action (Section 10) in brackets. Every flow ends with an activity row, and an audit row where money, consent or access changes.

F1. Initial owner onboarding. Our team creates the business in admin after the sales call (invite sent by email/SMS).

1. Owner opens invite, verifies email or phone code [`acceptInvite`].
2. S21: enters legal name, DBA, address, phone, timezone, tax rate, logo [`updateBusinessProfile`]. A default location and Texas jurisdiction are set.
3. S22 checklist shows: Payments (not started), Texting (not started), Import (scheduled by us), Legal policies (our job), Invite staff.
4. The shop can create customers and estimates immediately; sending uses email until texting is approved.

F2. Texting registration.

1. Owner opens S26 → Start. Form collects: legal business name, EIN (or sole-proprietor path), business type, address, website or Google Business URL, authorized contact name/title/email/phone, and attestations (copy from legal policy).
2. [`submitTextingRegistration`] stores the data, creates Twilio subaccount, Secondary Customer Profile, Brand; state `submitted`.
3. Worker polls and receives status callbacks; brand approved → campaign submitted with our standard use-case description and sample messages; state `pending`.
4. Number is purchased and attached to the shop's Messaging Service when the brand is approved; it cannot send until the campaign is approved.

5. Campaign approved → state `approved`; owner notified by email; S26 offers a test text to the owner's phone.
6. Rejection → state `rejected_action_required` with the provider reason in plain words; support notified; owner fixes and resubmits.

F3. Stripe merchant onboarding.

1. Owner opens S25 → Connect [`startStripeOnboarding`] creates a Standard connected account (if none) and an Account Link; owner completes Stripe-hosted onboarding.
2. Return URL shows "Checking with Stripe"; `account.updated` webhook updates `charges_enabled`, `payouts_enabled`, requirements.
3. Until `charges_enabled`, invoice pages hide Pay and show the shop phone.

F4. Concierge import (Section 15). Admin uploads files → dry run → reviews report with the owner on a call → commit → owner sees customers and Money Finder populate → admin can reverse within 30 days.

F5. Create customer. + New → Customer → name, mobile, email → duplicate check by normalized phone/email (warning with link, owner can continue) [`createCustomer`] → offer Add vehicle.

F6. Add vehicle. From customer → Add vehicle → VIN (decode on 17 valid characters) or year/make/model → plate, mileage [`createVehicle`, `recordMileage`]. Decode failure leaves fields editable.

F7. Create estimate. Customer → New estimate → pick vehicle (or add) → mileage in → add lines (type, description, qty, unit price, taxable default by type) or saved lines → photos → note → Save draft [`saveEstimateDraft`]. Totals computed server-side.

F8. Send estimate, texting active. Send → channel defaults to SMS if texting approved, phone present and not opted out; else email [`sendEstimate`]: freezes version, assigns number if first send, creates public link, renders PDF (async), sends message with link, schedules follow-ups if enabled.

F9. Send estimate, texting not active. Same, channel forced to email. If no email: show "Add an email or use Customer signs here". UI states why SMS is unavailable.

F10. Customer reviews estimate. Link opens P1 [`viewPublicEstimate`]: first view records `viewed_at` and an activity. Every line starts undecided; the customer taps Approve or Decline on each one.

F11. Approve some, decline others. Customer decides each line; the approved total updates live; Submit stays disabled until every line has a decision.

F12. Capture authorization. Customer types full name, signs, checks the acknowledgement, taps Submit [`submitAuthorization`]: one transaction stores the authorization (version, decisions per line, amounts, name, signature file, IP, user agent, time, policy version and text hash), sets estimate status, creates one recommendation per declined line, revokes further submissions on that version, cancels follow-ups, notifies shop. P2 shows the receipt. **In-shop variant:** "Customer signs here" on S12 opens P1 on the shop device in kiosk mode; authorization method `in_person`; staff user recorded.

F13. Convert approved work to invoice. S12 → Create invoice [`createInvoiceFromEstimate`]: copies exactly the approved lines, quantities and prices from the authorized version into a draft invoice. While draft, the shop may only reduce: remove a line, lower a quantity or lower a price. Nothing can be added or increased.

F13b. Supplement (more work found). On S12 or the draft invoice, "Add work" [`reviseEstimate`] creates a new draft version containing every line from the authorized version plus the new lines; any draft invoice is discarded. Allowed only while no invoice for this estimate has been sent. The shop sends the new version; the customer re-decides every line (none preselected) and signs again. The new authorization supersedes the old one, and the invoice is created from it. Lines declined earlier keep their existing recommendation: approved now → `won`; declined again → stays `open`, no duplicate.

F14. Send invoice. Send [`sendInvoice`]: assigns invoice number, freezes, creates public link, PDF, sends by SMS or email, schedules reminders.

F15. Customer pays by card. P3 → Pay [`createCheckoutSession`] → Stripe Checkout on the shop's account → redirect to P4 ("processing") → webhook `checkout.session.completed` [`applyStripeEvent`] creates payment `succeeded`, invoice `paid`, receipt sent, reminders cancelled, attribution job queued.

F16. Record cash/check. S13 → Record payment → S14 amount equals balance → [`recordManualPayment`] → invoice paid, receipt offered.

F17. Invoice reminder. Enabled by default. Unpaid sent invoice → reminders at days 3, 7, 14 after send, inside the send window. Each job re-checks invoice state before sending.

F18. Estimate follow-up. Enabled by default. Sent, unauthorized, unexpired estimate → follow-ups at days 2 and 5.

F19. Owner opens Money Finder. S1 shows the five cards computed from Section 13 queries (cached per request, no stale snapshots).

F20. Select recovery targets. S2 opens with nothing preselected. Owner selects items one by one or taps "Select all eligible"; ineligible items stay greyed with their reason.

F21. Edit recovery text. S3 shows category template with merge fields (`{first_name}` , `{vehicle}` , `{item}` , `{amount}` , `{shop_name}` , `{link}`); owner edits; legal footer appended (not editable).

F22. Send outreach. Confirm → [`sendRecoveryOutreach`] creates one batch, one outreach per recipient with rendered body and tracked link, enqueues sends respecting window, throttle and cooldown. S4 shows progress.

F23. Customer replies. Inbound SMS → conversation → the most recent outreach to that phone within 30 days gets an `interaction` (reply). Owner sees the reply tagged with its outreach.

F24. Opportunity becomes work. From the thread or S2 item, owner taps Re-quote (declined/open estimate) or New estimate / New appointment (lapsed/service due). The new estimate or appointment stores `source_outreach_id` . Approving a re-quoted declined line resolves the recommendation as `won` .

F25. Paid invoice becomes directly recovered revenue. Payment succeeded → attribution job evaluates rules (Section 12) → direct record with evidence → S5 updates.

F26. Influenced revenue. Same job: delivered outreach, no direct chain, paid invoice created within the influenced window → influenced record.

F27. View customer timeline. S8 Timeline: newest first, 50 per page, each row links to its record.

F28. Create/view today's appointment. Today → + → customer, vehicle, date, time, note [`saveAppointment`]. Appointment appears in Today/Tomorrow and on the customer timeline.

F29. STOP opt-out. Customer texts STOP (or a carrier-recognized variant) → Twilio blocks further sends and posts inbound → [`handleInboundSms`] records consent `opted_out` for that number, sends nothing extra (Twilio sends the confirmation), flags the thread, removes the number from all eligible lists, cancels queued messages. START reverses on Twilio and in our ledger.

F30. Failed SMS delivery. Status callback `undelivered/failed` → message `failed` with error code → thread shows failure; for estimate/invoice sends the owner gets an in-app alert with "Send by email instead". Error 21610 (unsubscribed) updates consent.

F31. Failed payment. Checkout failure stays inside Stripe's page; nothing recorded. Asynchronous failure (`payment_intent.payment_failed`) records a failed payment attempt on the invoice timeline; invoice stays unpaid.

F32. Webhook retry/replay. Every event stored first; processing failures retry with backoff; after 8 failures the event is `dead` and alerts; admin fixes cause and replays from I4. Applying an already-applied event is a no-op.

F33. Provider outage. Section 19. The shop sees a banner per affected capability; work that can wait is queued; nothing is reported as sent or paid until the provider confirms.

6. State machines

Each state lives in a `status` column guarded by a Postgres `CHECK` on allowed values and a single domain function per entity that performs transitions. A transition not listed here is rejected with `INVALID_TRANSITION` and nothing is written.

Estimate (header)

- States: `draft` , `sent` , `viewed` , `authorized` , `declined` , `expired` , `void`
- `draft → sent` (sendEstimate) · `sent → viewed` (first public view) · `sent|viewed → authorized` (submitAuthorization with ≥ 1 approved line) · `sent|viewed → declined` (submitAuthorization with all lines declined) · `sent|viewed → expired` (expiry job at `expires_at`) · `sent|viewed|expired → draft` (revise: new draft version; old version superseded) · authorized → draft (supplement, F13b: only while no invoice for the estimate has been sent; any draft invoice is discarded) · `draft|sent|viewed|expired → void` (voidEstimate)
- Rejected: `authorized → draft` once an invoice has been sent, anything from `void` , `declined → authorized` , `draft → authorized` , sending an estimate with zero lines or zero total.

Estimate version

- States: `draft` , `sent` , `superseded`
- `draft → sent` (freezes; immutable trigger engaged) · `sent → superseded` (a newer version is sent)
- Rejected: any update to lines, totals or photos of a `sent` or `superseded` version; two `draft` versions for one estimate; `superseded → sent` .

Estimate line decision (lines are immutable; this is the line's decision state, stored on the authorization)

- States: `pending` (no authorization yet), `approved`, `declined`
- `pending → approved|declined` only via the version's single authorization.
- Rejected: changing a decision after the authorization is stored; deciding a line on a superseded version.

Authorization

- States: `recorded`, superseded, `revoked_by_shop`
- Created as `recorded` (append-only) · recorded → superseded (a supplement version of the same estimate is authorized; evidence kept) · `recorded → revoked_by_shop` only when the shop voids the estimate before any invoice exists; the row and evidence are kept, never deleted.
- Rejected: a second authorization for the same version; authorization for a `draft` version; authorization after link revocation or expiry; editing evidence fields.

Invoice

- States: `draft`, `sent`, `paid`, `void`
- `draft → sent` (sendInvoice) · `sent → paid` (succeeded payments net of refunds ≥ total) · `paid → sent` (refund reduces paid amount below total; invoice shows balance due) · `draft|sent → void` (voidInvoice; reissue creates a new draft that references it)
- Rejected: editing lines after `draft`; any invoice line that adds work or exceeds the authorized quantity or price; `paid → void` (paid invoices are never voided in MVP; support handles exceptions); recording a payment on `draft` or `void`; `void → anything`.

Payment

- States: `pending`, `succeeded`, `failed`, `refunded`, `partially_refunded`, `disputed`
- Card: `pending → succeeded|failed` (Stripe events) · `succeeded → partially_refunded|refunded` (charge.refunded) · `succeeded|partially_refunded → disputed` (dispute created) · `disputed → succeeded|refunded` (dispute won/lost)
- Cash/check: created directly as `succeeded`; `succeeded → refunded` only by voiding the manual payment (Owner, audited, reason required).
- Rejected: negative or zero amounts; any amount change; `failed → succeeded` on the same payment (a retry is a new payment); manual payments with Stripe states.

Recommendation (declined work)

- States: `open`, `quoted`, `won`, `lost`, `dismissed`

- `open → quoted` (re-quote estimate sent containing a line linked to it) · `quoted → won` (that line approved) · `quoted → open` (re-quote declined, expired or voided) · `open|quoted → lost` (owner marks "done elsewhere") · `open → dismissed` (owner dismisses with reason) · `dismissed → open` (owner restores)
- Rejected: deleting a recommendation; changing amount, description or originating line; `won → anything`.

Message

- States: `queued`, `scheduled`, `sending`, `sent`, `delivered`, `undelivered`, `failed`, `cancelled`, `received` (inbound only)
- Outbound: `queued|scheduled → sending → sent → delivered|undelivered` · `sending → failed` · `queued|scheduled → cancelled` (state no longer valid, opt-out, owner cancel)
- Rejected: sending without a send-eligibility check passing at send time; `delivered → anything` except recording a late `undelivered` callback (out-of-order handling, Section 19); editing body after `sending`.

Conversation

- States: `open`, `archived`; flags `unread`, `opted_out`
- `open ↔ archived` (owner) · new inbound message → `open` + unread
- Rejected: a second conversation for the same business + phone + channel; sending into a conversation whose phone is opted out.

Texting registration

- States: `not_started`, `information_required`, `submitted`, `pending`, `approved`, `rejected_action_required`, `suspended`
- `not_started → information_required` (owner opens form) · `information_required → submitted` (Owner submits valid data) · `submitted → pending` (brand approved, campaign under review) · `submitted|pending → rejected_action_required` (provider rejection) · `rejected_action_required → submitted` (resubmit) · `pending → approved` (campaign verified and number attached) · `approved → suspended` (provider suspends number or campaign) · `suspended → pending|approved` (resolved)
- Rejected: sending SMS in any state but `approved`; Staff submitting; `not_started → approved`.

Recovery opportunity (a computed item made durable when acted on)

- States: `eligible`, `selected`, `contacted`, `responded`, `converted`, `dismissed`, `snoozed`, `resolved`
- Live lists are computed; a row is persisted when the owner selects, dismisses or snoozes an item. `selected → contacted` (outreach sent) · `contacted → responded` (interaction) · `responded|contacted → converted` (linked estimate or appointment created) · `any open → resolved` (underlying item resolved: paid, approved, customer returned) · `eligible → dismissed|snoozed` · `snoozed → eligible` (snooze ends)
- Rejected: contacting a dismissed item; re-contacting inside the cooldown (Section 14).

Recovery outreach (one per recipient)

- States: `queued`, `sent`, `delivered`, `failed`, `cancelled`; plus interaction flags `replied`, `clicked`, `approved_via_link`, `paid_via_link`
- Follows its message's status; interactions recorded any time within 30 days.
- Rejected: interactions recorded after 30 days; editing the rendered body after send.

Appointment

- States: `scheduled`, `completed`, `cancelled`, `no_show`
- `scheduled → completed|cancelled|no_show` · `cancelled → scheduled` (restore)
- Rejected: `completed → scheduled`; appointments with no customer.

Import job

- States: `uploaded`, `parsing`, `validated`, `dry_run_ready`, `committing`, `committed`, `failed`, `reversing`, `reversed`
- `uploaded → parsing → validated → dry_run_ready → committing → committed` · any running state `→ failed` · `committed → reversing → reversed` (within 30 days, only if reversal check passes)
- Rejected: commit without a dry run; commit of a dry run older than 24 hours or of changed files; second concurrent job for the same business; reversal after records were used in estimates, invoices or outreach (those records are kept; reversal is partial and reported).

7. Domain model

53 tables: 46 tenant-owned (including request_idempotency, Section 10), 7 global (business, user, internal_user, support_session, vin_decode_cache, webhook_event, legal_policy). Nothing exists here only because the long-term strategy mentions it; each

table is used by an MVP flow or protects a foundation.

Standard columns on every tenant table: `id` (UUIDv7, PK), `business_id` (NOT NULL, FK), `created_at`, `updated_at`, `created_by_user_id` (nullable for system/public actions), plus `UNIQUE (business_id, id)` so other tables can reference it with a composite key. Money columns are `bigint` cents. Timestamps are `timestamptz` UTC; display uses the business timezone. In the table, fields are required unless marked *opt*.

Tenancy and access

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
business (global root, is the tenant)	The shop	legal_name, dba *opt*, phone, address fields, timezone, jurisdiction_code (`US-TX`), tax_rate_bps, logo_file_id *opt*, status (`active`, `suspended`, `closed`)	Has locations, memberships, everything	—	Never hard-deleted in MVP; `closed` + support-run purge later (NOT NOW)	Create, status change, profile changes
location	Structural only (D4)	name, address copied from business, is_default	Belongs to business	`UNIQUE (business_id) WHERE is_default`	Created with business; not user-visible	—
user (global)	A login identity	auth_provider_id, email *opt*, phone *opt*, display_name, mfa_phone *opt*	Has memberships	Unique auth_provider_id, unique email, unique phone	Deactivate, never delete while referenced	Sign-in, MFA changes
membership	User ↔ business with role	user_id, role (`owner`, `staff`), status (`invited`, `active`, `removed`)	Business, user	`UNIQUE (business_id, user_id)`; `UNIQUE (business_id) WHERE role='owner' AND status='active'`; max 2 active enforced by trigger	Removed, never deleted	All changes
support_session (global)	View-as record	admin_user_id, business_id, reason, started_at, expires_at, ended_at *opt*	Business	Index (business_id, started_at)	Append-only	Is itself audit; also mirrored to audit_event
internal_user (global)	Our staff for /admin	email, role (`support`, `admin`), active	—	Unique email	Deactivate	All changes

Customers and vehicles

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
customer	The person the shop serves	first_name, last_name *opt*, phone_e164 *opt*, email *opt*, notes_summary *opt*, do_not_contact (bool), status (`active`, `archived`, `deleted`), source (`manual`, `import`, `inbound_sms`), import_job_id *opt*, last_visit_at (derived, maintained)	Vehicles, estimates, invoices, notes	Index (business_id, phone_e164); (business_id, lower(email)); trigram on name; *no* uniqueness on phone (shared phones are real)	Archive hides; Owner delete = soft delete only when no estimates, invoices or payments exist, else blocked	Delete, do_not_contact changes
asset	Generic asset (D3 strategy)	customer_id, asset_type (`vehicle` only in MVP), display_name, status (`active`, `sold`, `inactive`), next_service_on *opt*, next_service_description *opt*, next_service_amount_cents *opt*	Customer; 1:1 extension	Index (business_id, customer_id)	Status change only; delete blocked when referenced	Status changes
vehicle	Vehicle extension of asset	asset_id (PK and FK), vin *opt*, year *opt*, make *opt*, model *opt*, trim *opt*, engine *opt*, plate *opt*, plate_state *opt*, decode_source *opt*	Asset	Index (business_id, vin); (business_id, upper(plate)); (business_id, right(vin, 6))	With asset	—
asset_reading	Odometer history	asset_id, reading_type (`odometer`), value (integer), unit (`mi`), read_on, source (`manual`, `estimate`, `invoice`, `import`)	Asset	Index (asset_id, read_on desc)	Append-only; corrections are new rows with source `manual`	—
note	Free-text memory	subject_type (`customer`, `vehicle`), subject_id, body	Customer or vehicle (composite FK per type via two nullable columns)	Index (business_id, customer_id, created_at)	Edit allowed (history kept in activity); soft delete	—
vin_decode_cache (global)	Avoid repeat lookups	vin, payload, decoded_at	—	Unique vin	Refresh after 180 days	—

Estimates, authorization, declined work

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
estimate	Header and current state	location_id, customer_id, asset_id *opt*, number *opt until first send*, status, current_version_id, expires_at *opt*, source_outreach_id *opt*, source_recommendation_ids (via line link)	Versions, invoice	`UNIQUE (business_id, number)`; index (business_id, status, expires_at)	Void, never delete once sent; drafts deletable	Send, void, revise

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
estimate_version	Exact document sent	estimate_id, version_no, status, mileage_in *opt*, customer_note *opt*, subtotal_cents, taxable_cents, tax_rate_bps, tax_cents, total_cents, sent_at *opt*, sent_channel *opt*, snapshot_hash *opt*, pdf_file_id *opt*, policy_version_id (authorization text in force)	Estimate, lines, photos	`UNIQUE (estimate_id, version_no)`; one draft per estimate	Immutable after `sent` (trigger)	Send
estimate_line	A line on one version	estimate_version_id, position, line_type (`labor`, `part`, `fee`), description, quantity_milli (integer, 1000 = 1.0), unit_price_cents, taxable, line_total_cents, recommendation_id *opt* (set when re-quoting declined work)	Version	`UNIQUE (estimate_version_id, position)`	Immutable with version	—
estimate_photo	Photo on a version	estimate_version_id, file_id, position, caption *opt*	Version, file	`UNIQUE (estimate_version_id, position)`	Immutable with version; copied by reference to the next version	—
saved_line	Quick-add line	line_type, description, unit_price_cents, taxable, position	Business	Max 25 per business (trigger)	Delete allowed	—
authorization	Customer decision evidence	estimate_id, estimate_version_id, method (`online`, `in_person`), signer_name, signature_file_id, signed_at, ip_address (inet) *opt for in_person*, user_agent *opt*, public_link_id *opt*, staff_user_id *opt*, policy_version_id, authorization_text_sha256, approved_total_cents, declined_total_cents, status, sms_consent_given (bool) *opt*	Version, decisions	`UNIQUE (estimate_version_id)`	Append-only; only status may change to `revoked_by_shop`	Always (creation and revocation)
authorization_decision	Per-line decision	authorization_id, estimate_line_id, decision (`approved`, `declined`), line_total_cents (copied)	Authorization, line	`UNIQUE (authorization_id, estimate_line_id)`	Append-only	With authorization

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
recommendation	Declined work, first-class	customer_id, asset_id *opt*, origin_estimate_id, origin_version_id, origin_line_id, origin_authorization_id, description, amount_cents, declined_on, status, resolution_line_id *opt*, resolved_at *opt*, dismissed_reason *opt*	Line, authorization, customer, vehicle	`UNIQUE (business_id, origin_line_id)`; index (business_id, status, declined_on)	Never deleted; status changes only	Status changes

Invoices and payments

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
invoice	Bill for approved work	location_id, customer_id, asset_id *opt*, estimate_id, authorization_id, number *opt until sent*, status, subtotal_cents, taxable_cents, tax_rate_bps, tax_cents, total_cents, paid_cents (maintained), refunded_cents (maintained), sent_at *opt*, paid_at *opt*, replaces_invoice_id *opt*, pdf_file_id *opt*, mileage_out *opt*, source_outreach_id *opt* (copied from estimate)	Estimate, authorization, lines, payments	`UNIQUE (business_id, number)`; index (business_id, status, sent_at)	Immutable after `sent` except status and maintained paid/refunded columns; void, never delete once sent	Send, void, status
invoice_line	Line on invoice	invoice_id, position, line_type, description, quantity_milli, unit_price_cents, taxable, line_total_cents, estimate_line_id *(required: an approved line of the invoice's authorization)*	Invoice	`UNIQUE (invoice_id, position)`	Immutable after send	Changes while draft logged to activity
payment	Money received	invoice_id, method (`card`, `cash`, `check`), status, amount_cents, received_at, check_number *opt*, stripe_account_id *opt*, stripe_checkout_session_id *opt*, stripe_payment_intent_id *opt*, stripe_charge_id *opt*, failure_code *opt*, voided_reason *opt*	Invoice	Unique (stripe_payment_intent_id) where not null; index (business_id, received_at)	Never deleted	Always
payment_refund	Refund mirrored from Stripe	payment_id, stripe_refund_id, amount_cents, refunded_at, status	Payment	Unique stripe_refund_id	Append-only	Always
payment_dispute	Dispute mirrored from Stripe	payment_id, stripe_dispute_id, amount_cents, status, reason, evidence_due_by *opt*	Payment	Unique stripe_dispute_id	Status updates only	Always

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
business_sequence	Number generators	kind (`estimate` , `invoice`), next_value	Business	`UNIQUE (business_id, kind)`	Row-locked increment	—

Messaging and consent

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
texting_account	Registration and provider state	state (Section 6), twilio_subaccount_sid *opt*, messaging_service_sid *opt*, customer_profile_sid *opt*, brand_sid *opt*, campaign_sid *opt*, brand_type (`standard` , `low_volume_standard` , `sole_proprietor`), submitted_data (jsonb, encrypted EIN), rejection_reason *opt*, last_synced_at	Business	`UNIQUE (business_id)`	State changes only	All state changes and submissions
phone_number	The shop's number	texting_account_id, location_id, e164, provider_sid, status (`provisioned` , `active` , `released`)	Texting account	Unique e164 globally among non-released	Released only by admin	Always
conversation	Thread per phone	channel (`sms`), counterpart_e164, status, unread_count, last_message_at, opted_out (bool, mirrored from consent)	Messages; linked customers via `customer.phone_e164` match	`UNIQUE (business_id, channel, counterpart_e164)`	Archive only	—
message	Every SMS and email we send or receive	conversation_id *opt for email*, customer_id *opt*, channel (`sms` , `email`), direction, purpose (`transactional` , `reminder` , `recovery` , `reply` , `system`), status, body, to_address, from_address, provider_message_id *opt*, error_code *opt*, scheduled_for *opt*, sent_at *opt*, delivered_at *opt*, related_type/related_id *opt* (estimate, invoice, outreach), idempotency_key, segments *opt*	Conversation, customer	`UNIQUE (business_id, idempotency_key)` ; unique provider_message_id; index (conversation_id, created_at)	Never deleted; body retained per retention policy	Purpose `recovery` sends audited via batch

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
consent_event	Append-only consent ledger	channel, address (phone E.164 or email), event (`opt_in` , `opt_out` , `opt_in_restored`), scope (`transactional` , `recovery` , `all`), source (`sms_keyword` , `authorization_checkbox` , `owner_recorded` , `import_attested` , `email_unsubscribe` , `provider_error`), evidence (jsonb), occurred_at	Business	Index (business_id, channel, address, occurred_at desc)	Append-only	Is audit-grade by design
consent_state	Current state per address (derived)	channel, address, transactional_allowed, recovery_allowed, opted_out_at *opt*	Business	`UNIQUE (business_id, channel, address)`	Rebuilt from events by trigger	—
message_template	Editable copy	kind (`estimate_sent` , `invoice_sent` , `estimate_followup` , `invoice_reminder` , `receipt` , `recovery_declined` , `recovery_open_estimate` , `recovery_unpaid` , `recovery_lapsed` , `recovery_service_due`), channel, body, is_default	Business; system defaults global	`UNIQUE (business_id, kind, channel)`	Edit, reset	—
automation_setting	The two reminders	kind (`invoice_reminder` , `estimate_followup`), enabled	Business	`UNIQUE (business_id, kind)`	Toggle	Toggle
scheduled_send	A future reminder step	kind, target_type, target_id, step (1, 2, 3), run_at, status (`scheduled` , `done` , `skipped` , `cancelled`), message_id *opt*	Target	`UNIQUE (business_id, kind, target_id, step)`	Terminal states kept 1 year	—
email_suppression	Hard bounces and complaints	email, reason, occurred_at	Business	`UNIQUE (business_id, lower(email))`	Removed only by admin	Always

Money recovery

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
recovery_item	Durable state of an opportunity once touched	category (`declined_work`, `open_estimate`, `unpaid_invoice`, `lapsed`, `service_due`), subject_type, subject_id (recommendation, estimate, invoice, customer, asset), customer_id, status, value_cents_at_selection *opt*, snoozed_until *opt*, dismissed_reason *opt*, last_outreach_at *opt*	Subject, customer	`UNIQUE (business_id, category, subject_id)`	Status changes; never deleted	Dismiss and restore
outreach_batch	One owner send action	category, created_by_user_id, template_body (as edited), recipient_count, channel_counts (jsonb)	Outreaches	Index (business_id, created_at)	Immutable	Always
outreach	One recipient in a batch	batch_id, recovery_item_id, customer_id, channel, message_id, tracking_token_hash, status, interaction flags, first_interaction_at *opt*	Batch, item, message	Unique tracking_token_hash; index (business_id, customer_id, created_at)	Immutable except status/flags	—
outreach_interaction	Evidence of a customer response	outreach_id, kind (`reply`, `link_click`, `estimate_approved_via_link`, `invoice_paid_via_link`, `owner_marked_response`), occurred_at, evidence_ref (message id, authorization id, payment id)	Outreach	Index (outreach_id, occurred_at)	Append-only	owner_marked_response audited
attribution	A counted dollar amount	kind (`direct`, `influenced`), outreach_id, invoice_id, amount_cents, rule_code, evidence (jsonb chain), explanation (text), status (`active`, `reversed`), reversed_reason *opt*, is_manual	Outreach, invoice	`UNIQUE (business_id, invoice_id) WHERE status='active'`	Reversal only	Always

Records, history, platform

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
activity	Customer timeline	customer_id *opt*, asset_id *opt*, kind (enumerated list, e.g. `estimate.sent`), subject_type, subject_id, actor_type (`user`, `customer`, `system`, `support`), actor_id *opt*, summary (short text, no secrets), data (jsonb)	Anything	Index (business_id, customer_id, occurred_at desc)	Append-only	—
audit_event	Security and money record	actor_type, actor_id, action, target_type, target_id, before (jsonb) *opt*, after (jsonb) *opt*, ip *opt*, request_id, support_session_id *opt*	Anything	Index (business_id, occurred_at); (target_type, target_id)	Append-only; table grants deny UPDATE/DELETE to the app role	—
file	Stored object	storage_key, kind (`estimate_photo`, `signature`, `estimate_pdf`, `invoice_pdf`, `authorization_pdf`, `logo`, `import_source`, `export`), content_type, byte_size, sha256, status (`pending`, `ready`, `quarantined`)	Referenced by others	Unique storage_key	Delete only when unreferenced and not tied to a sent document	Deletion
public_link	Tokenized access	token_hash (sha256), target_type (`estimate_version`, `invoice`), target_id, expires_at, revoked_at *opt*, view_count, last_viewed_at *opt*	Target	Unique token_hash; index (target_type, target_id)	Revoke; never delete	Revocation
appointment	Today/tomorrow schedule	location_id, customer_id, asset_id *opt*, starts_at, note *opt*, status, estimate_id *opt*, source_outreach_id *opt*	Customer	Index (business_id, starts_at)	Status changes; delete allowed for `cancelled`	—
historical_sale	Imported past sale (D11)	customer_id, sold_on, amount_cents, description *opt*, source (`square`, `quickbooks`, `csv`), source_ref, import_job_id	Customer, import job	`UNIQUE (business_id, source, source_ref)`	Deleted only by import reversal	Reversal
import_job	Concierge import run	source_profile, status, file_ids, dry_run_report (jsonb), committed_counts (jsonb), started_by_internal_user_id, committed_at *opt*, reversed_at *opt*	Rows	One active job per business (partial unique on non-terminal status)	Kept	Commit, reversal
import_row	Staged row	import_job_id, row_number, entity (`customer`, `vehicle`, `sale`), raw (jsonb), normalized (jsonb), status (`valid`, `warning`, `error`, `duplicate`, `committed`, `skipped`), messages (jsonb), target_id *opt*, source_key	Job	`UNIQUE (import_job_id, row_number)`; `UNIQUE (business_id, entity, source_key) WHERE status='committed'`	Purged 90 days after commit (raw contains PII)	—
stripe_account	Connect state	stripe_account_id, charges_enabled, payouts_enabled, details_submitted, requirements (jsonb), last_synced_at	Business	`UNIQUE (business_id)`; unique stripe_account_id	Disconnect by admin only	All changes

Entity	Purpose	Key fields	Relationships	Indexes / uniqueness	Lifecycle and deletion	Audit
webhook_event (global, tenant tag)	Inbox for every provider callback	provider (`stripe`, `twilio`, `postmark`), provider_event_id, event_type, business_id *opt* (resolved), payload (jsonb), signature_valid, status (`received`, `processing`, `applied`, `ignored`, `failed`, `dead`), attempts, last_error *opt*, received_at, applied_at *opt*	Business when resolved	`UNIQUE (provider, provider_event_id)`	Kept 1 year, then payload trimmed	Replays audited
legal_policy (global)	Attorney-approved copy	jurisdiction_code, kind (`estimate_authorization`, `invoice_terms`, `sms_consent_disclosure`, `recovery_footer`, `retention_rules`, `texting_attestation`), version, body, status (`draft`, `approved`, `retired`), approved_by, approved_at, approval_reference (attorney memo id)	Referenced by versions and authorizations	`UNIQUE (jurisdiction_code, kind, version)`; one approved per (jurisdiction, kind)	Retire, never delete	Approvals

Not created in MVP (expansion paths noted): `contact` (move phone/email off customer later), `address`, `payment_allocation` (D8), `invoice_version` (D7), `job`, `tag`, `subscription` / `entitlement` (our billing is manual), `service_catalog`.

8. Database constraints

The database enforces every invariant below; application code may check first for nicer errors, but correctness never depends on it. Each one has a constraint test (Section 22).

Invariant	Enforcement
Cross-tenant references impossible	Every tenant table has `UNIQUE (business_id, id)`; every FK between tenant tables is composite: `FOREIGN KEY (business_id, customer_id) REFERENCES customer (business_id, id)`. RLS on every tenant table (Section 9)
Phone numbers normalized	Stored only as E.164: `CHECK (phone_e164 ~ '^\+[1-9][0-9]{7,14}$')`. Normalization happens in one server function using libphonenumber with default region US; invalid input is rejected, never stored raw

Invariant	Enforcement
Emails normalized	Stored lowercased and trimmed; `CHECK (email = lower(btrim(email)))`
Money is integer and non-negative	All money columns `bigint`; `CHECK (x >= 0)` on line prices, totals, payment and refund amounts; payment `CHECK (amount_cents > 0)`
Totals are consistent	Trigger on sent version and invoice verifies `subtotal = sum(lines)`, `tax = round_half_up(taxable * rate / 10000)`, `total = subtotal + tax` at the moment of send; mismatched sends fail
Sent versions immutable	`BEFORE UPDATE OR DELETE` trigger on estimate_version, estimate_line and estimate_photo raises when the parent version status is `sent` or `superseded`, except the one allowed `sent → superseded` status change
Sent invoices immutable	Trigger allows only status, paid_cents, refunded_cents, paid_at changes after `sent`; invoice_line changes blocked after send
One draft version per estimate	Partial unique index `ON estimate_version (estimate_id) WHERE status = 'draft'`
Authorization only for a sent version	Trigger on insert checks referenced version status is `sent` and the estimate is not void; `UNIQUE (estimate_version_id)`
Authorization decisions belong to that version	`authorization_decision` references `(business_id, estimate_line_id)`; trigger checks the line's version equals the authorization's version; count of decisions must equal count of lines (deferred constraint trigger checked at commit)

Invariant	Enforcement
Evidence is append-only	App role has `INSERT, SELECT` only on authorization, authorization_decision, audit_event, consent_event, activity, outreach_interaction; status column of authorization updatable through a SECURITY DEFINER function only
Declined recommendation tied to its line	recommendation FK to `(business_id, origin_line_id)`; `UNIQUE (business_id, origin_line_id)`; trigger checks the origin decision is `declined`
Invoice comes from an authorized estimate	`invoice.authorization_id` NOT NULL and FK; trigger checks authorization status `recorded`. Invoice cannot exceed authorization: every invoice_line must reference an approved line of that authorization's version, at most once, with quantity and unit price no greater than authorized (reductions only); no invoice line without an estimate line
Payments cannot overpay	Trigger on payment insert/state change: sum of `succeeded` payments minus refunds ⩽ invoice total; violation raises (`exception: a card payment arriving by webhook is always recorded because the money already moved; if it overpays, it is stored with an overpayment flag and support is alerted to refund the difference in Stripe;` Section 19)
One Stripe intent, one payment	Unique `stripe_payment_intent_id`, `stripe_refund_id`, `stripe_dispute_id`
One webhook applied once	`UNIQUE (provider, provider_event_id)` on webhook_event; processing updates status with `WHERE status IN ('received','failed')` and row lock
One message per idempotency key	`UNIQUE (business_id, idempotency_key)` on message; key format `{purpose}:{target}:{step}` or `{outreach_id}`

Invariant	Enforcement
One reminder step once	`UNIQUE (business_id, kind, target_id, step)` on scheduled_send
Sequential numbers unique	`UNIQUE (business_id, number)` on estimate and invoice; numbers drawn from business_sequence with `SELECT … FOR UPDATE` inside the send transaction; gaps allowed only from rolled-back sends (acceptable, documented)
Public tokens unique and unguessable	Store only `sha256(token)`; `UNIQUE (token_hash)`; token is 32 random bytes, base64url
One conversation per phone	`UNIQUE (business_id, channel, counterpart_e164)`
Consent state follows events	consent_state maintained only by trigger on consent_event insert; app role has no UPDATE on consent_state
STOP blocks sending	`BEFORE INSERT` trigger on message for outbound SMS rejects when consent_state shows opted out for that number (system purpose `stop_confirmation` excepted; Twilio sends that natively so we do not)
One active attribution per invoice	Partial unique index on attribution `(business_id, invoice_id) WHERE status = 'active'`
Max one owner, max two members	Partial unique index for owner; trigger blocks a third active membership
One active import per business	Partial unique index on import_job `(business_id) WHERE status NOT IN ('committed','failed','reversed')`
Status values valid	`CHECK (status IN (...))` on every status column; transitions enforced by domain functions (Section 6)

Invariant	Enforcement
Audit cannot be edited	Revoke UPDATE, DELETE, TRUNCATE on audit_event from all app roles

9. Tenant security model

A request touches tenant data only inside a transaction that has proven, server-side, which business it belongs to. Four layers must all fail before data leaks.

Session → membership → active business

1. Supabase Auth issues the session cookie (httpOnly, Secure, SameSite=Lax). The server verifies it on every request; the browser never sends a business ID we trust.
2. Server loads the user's active memberships. With one membership (the MVP norm), that is the active business. With more than one, the active business is stored server-side on the session record and switched only through `switchBusiness`, which re-checks membership.
3. Removed or suspended memberships end the session on next request.

Database tenant context

- The app connects as role `app_user`: no `BYPASSRLS`, no superuser, no table ownership.
- Every unit of work runs as `withTenant(businessId, actor, fn)`: opens a transaction, runs `SET LOCAL app.business_id = …`, `SET LOCAL app.actor_id = …`, `SET LOCAL app.actor_type = …`, then `fn`. `SET LOCAL` dies with the transaction, so pooled connections never carry context forward. Use transaction-mode pooling only with this pattern.
- Queries outside `withTenant` against tenant tables return zero rows (RLS default deny).

RLS expectations

- Every tenant table: `ENABLE` and `FORCE ROW LEVEL SECURITY`; one policy `USING (business_id = current_setting('app.business_id', true)::uuid)` with the same `WITH CHECK`.
- Global tables (`user`, `legal_policy`, `vin_decode_cache`) have explicit read policies; `webhook_event` and `internal_user` are not readable by `app_user` at all (worker uses role `worker_user`, admin uses `admin_user`, both still subject to tenant RLS on tenant tables).

- Migrations run as `migration_owner`, the only role that owns tables.
- A CI check fails the build if any table with a `business_id` column lacks RLS enabled + forced + policy.

Data access layer

- Repositories receive a tenant-scoped transaction object, not a raw pool. A lint rule forbids importing the raw database client outside `db/tenant.ts` and `db/admin.ts`.
- Every query still includes `business_id` explicitly (belt and braces; helps indexes).
- Domain functions never accept a business ID from input; they read it from the transaction context.

Composite foreign keys: as Section 8. A bug that tries to attach Business B's customer to Business A's estimate fails at the FK even if RLS were misconfigured.

Support view-as

- Admin starts a session in `/admin` with reason; server creates `support_session` (60 minutes), then serves the shop UI read-only: all mutating server actions check `actor_type = 'support'` and reject.
- Every read during view-as runs under the shop's tenant context with `actor_type = 'support'` and writes one audit event at start and end (not per page).
- The owner sees the session in S29.

Public link isolation

- Public routes never have a session. The token is hashed and resolved by a SECURITY DEFINER function `resolve_public_link(token_hash)` that returns only `(business_id, target_type, target_id, status)` and is the only public-path query that runs without tenant context.
- The page then runs inside `withTenant(business_id, actor_type='customer')` and may read only the target document and the fields listed in Section 16, through a dedicated read model, not the shop repositories.
- Unknown, expired and revoked tokens return the same response and timing class.

File isolation

- Private bucket; object keys prefixed `b/{business_id}/…` (Section 17); the app serves files only by file ID after a tenant-scoped lookup, issuing 5-minute signed URLs. Storage bucket policies deny all public access.

Worker isolation: every job payload carries `business_id`; the worker runs each job inside `withTenant`. A job without a business ID may touch only global tables (webhook routing, cache).

Cross-tenant test matrix (runs on every pull request; Section 22)

Attack	Expected result
User of Business A requests B's customer, estimate, invoice, vehicle, conversation, file by ID through every server action	Not found (404 semantics), no data in response or logs
Mutation in A referencing B's IDs (estimate for B's customer, payment on B's invoice, outreach to B's recovery item)	FK violation or not found; nothing written
Raw query without tenant context	Zero rows
`withTenant(A)` query with explicit `WHERE business_id = B`	Zero rows
Public token for A's estimate used to fetch A's other documents or B's anything	Rejected
Signed URL for A's photo requested by a B session	Not issued
Worker job for A with payload IDs from B	Fails, alerts
Inbound SMS to A's number from a phone that is B's customer	Lands only in A's conversation
Stripe webhook with B's account ID and metadata claiming A's invoice	Rejected: account ID must match the invoice's business
Staff of A calls an Owner-only action	Forbidden
Support view-as attempts a mutation	Forbidden, audited

10. Server action contracts

Internal server actions and route handlers, not a public API. The contracts below are binding: inputs, authorization, transaction boundary, side effects, activity, audit, idempotency and errors.

Rules that apply to every action

- **Validation:** Zod schema per action; strings trimmed; unknown fields rejected; money as integer cents; phone normalized by the shared function.
- **Authorization:** caller resolved from session (Section 9). "Shop user" means Owner or Staff with an active membership and not in support view-as. Owner-only actions say so.
- **Transaction:** one `withTenant` transaction per action unless stated. External calls (Stripe, Twilio, email, VIN) never run inside a database transaction; the action writes intent rows and enqueues a job, or calls the provider before/after with compensating state.
- **Idempotency:** every mutating action takes `clientRequestId` (UUID generated when the form opens). Stored in `request_idempotency (business_id, client_request_id, action, response_json, created_at)`, unique per business; a repeat returns the stored response. Kept 48 hours.
- **Errors:** typed codes: `VALIDATION`, `NOT_FOUND`, `FORBIDDEN`, `INVALID_TRANSITION`, `CONFLICT` (stale version; carries current state), `PROVIDER_UNAVAILABLE`, `NOT_ELIGIBLE` (messaging), `RATE_LIMITED`. Messages are plain words; codes drive UI.
- **Concurrency:** updates to drafts carry `expectedUpdatedAt`; mismatch returns `CONFLICT`.

Shop and team

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Errors
acceptInvite	Invitee (authenticated)	invite token	Activate membership; create user if new	Audit: membership.activated	Expired/used token
inviteStaff	Owner	name, email or phone	Create invited membership (blocks if 2 active); send invite email/SMS (system purpose)	Audit: membership.invited	`CONFLICT` if slot full
removeStaff	Owner	membership_id	Status `removed`; revoke sessions	Audit	Cannot remove owner

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Errors
updateBusinessProfile	Shop user	profile fields; tax_rate_bps 0–2000	Update business and default location address	Audit with before/after	—
saveSavedLine / deleteSavedLine	Shop user	type, description ≤ 200, price ≥ 0	Upsert; cap 25	—	`CONFLICT` at cap
setAutomation	Shop user	kind, enabled	Toggle; disabling cancels scheduled steps of that kind	Audit	—
updateTemplate / resetTemplate	Shop user	kind, channel, body ≤ 480 chars SMS, merge fields from allowed list	Upsert	Audit	`VALIDATION` on unknown merge field

Customers, vehicles, notes, schedule

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Errors
createCustomer	Shop user	first_name (req), last_name, phone, email	Insert; returns duplicate candidates (not blocking)	Activity customer.created	`VALIDATION` invalid phone/email
updateCustomer	Shop user	fields, expectedUpdatedAt	Update; phone change does not move past conversations	Activity customer.updated (fields named, values not)	`CONFLICT`
archiveCustomer / restoreCustomer	Shop user	id	Status change	Activity	—
deleteCustomer	Owner	id	Soft delete only if no estimates, invoices, payments	Audit	`CONFLICT` with reason
setDoNotContact	Shop user	id, value	Flag; excludes from Money Finder and reminders	Activity, audit	—
recordConsent	Shop user	customer_id, scope, source `owner_recorded`, note (how consent was obtained)	Insert consent_event	Audit	Phone missing
addNote / editNote / deleteNote	Shop user	subject, body ≤ 5,000	Insert/update/soft delete	Activity note.added	—
createVehicle / updateVehicle	Shop user	customer_id, VIN (17 chars, check digit validated; mismatch is a warning, not a block), year 1950–next year, make, model, plate	Insert asset + vehicle; enqueue VIN decode if VIN present and fields empty	Activity vehicle.added	—

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Errors
decodeVin	Shop user	vin	Cache lookup, else provider call (outside transaction, 3 s timeout); returns fields to the form, does not save	—	`PROVIDER_UNAVAILABLE` (form stays editable)
recordMileage	Shop user	asset_id, value, read_on	Insert reading; warn if lower than latest	Activity	—
setNextService	Shop user	asset_id, date, description, amount (opt)	Update asset	Activity vehicle.next_service_set	Date in past beyond 90 days
saveAppointment	Shop user	customer_id, asset_id, starts_at, note, source_outreach_id (opt, from thread)	Insert/update	Activity appointment.scheduled	—
setAppointmentStatus	Shop user	id, status	Transition (Section 6)	Activity	`INVALID_TRANSITION`
search	Shop user	q (≥ 2 chars)	Read-only; ≤ 20 results per group	—	—

Estimates and authorization

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
saveEstimateDraft	Shop user	customer_id, asset_id, mileage_in, lines[] (≤ 100; qty > 0; price ≥ 0; description ≤ 500), photo file IDs (≤ 10), note, expectedUpdatedAt	Create estimate + draft version, or replace draft lines; server computes totals	Activity estimate.drafted (first save only)	`CONFLICT`
uploadEstimatePhoto	Shop user	file (Section 17)	Direct-to-storage upload with signed URL; file row `pending` → `ready` after processing job	—	Type/size errors
sendEstimate	Shop user	estimate_id, channel (`sms` , `email` , `none` for in-shop), recipient override (opt)	Tx: assign number (first send), freeze version (`sent` , snapshot hash), supersede previous sent version and revoke its link, create public link (expires at estimate expiry + 7 days), set expires_at = now + 30 days (policy-configurable), create message row (queued), schedule follow-ups if enabled, enqueue PDF render. After commit: worker sends message	Activity estimate.sent; audit estimate.sent	Requires ≥ 1 line, total > 0, approved legal policy, channel eligible; `NOT_ELIGIBLE` explains channel
reviseEstimate	Shop user	estimate_id	Create new draft version copying lines and photos; estimate status `draft` (old sent version stays valid until the new one is sent)	Activity	After authorization: allowed only while no invoice for the estimate has been sent (supplement, F13b); discards a draft invoice. Not allowed if void

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
voidEstimate	Shop user	estimate_id, reason	Status `void`; revoke links; cancel follow-ups; if authorized and no invoice, authorization → `revoked_by_shop`	Audit	Not allowed if an invoice exists
startInShopSigning	Shop user	estimate_id	Creates a 15-minute public link flagged `in_person`, opens P1 on device	Audit	Version must be sent (channel `none` allowed)
viewPublicEstimate	Public (token)	token	Resolve link; record first view (status `viewed`, activity) and increment view count; return read model	Activity estimate.viewed (first time)	Uniform unavailable response
submitAuthorization	Public (token) or in-shop link	token, decisions for every line, signer_name (2–100 chars), signature image (PNG ⩽ 200 KB), acknowledgement = true, sms_consent (opt, only if policy offers it), clientRequestId	One tx: lock version; verify sent, not superseded, not expired, link valid, no existing authorization; store signature file reference; insert authorization + decisions with IP, user agent, policy version, text hash; estimate → `authorized` or `declined`; create recommendations for declined lines that don't already carry one (supplement lines keep theirs, which stays open); mark any earlier authorization of this estimate superseded; resolve recommendations linked to approved lines (`won`) or re-open those declined (`open`); record outreach interaction `estimate_approved_via_link` if link reached via outreach token; cancel follow-ups; consent event if given. After commit: render authorization PDF, notify shop	Activity estimate.authorized; audit authorization.recorded	Idempotent on clientRequestId and unique version constraint; `CONFLICT` if already authorized (returns existing receipt)

Invoices and payments

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
createInvoiceFromEstimate	Shop user	estimate_id	Draft invoice with exactly the approved lines, quantities and prices, each linked to its estimate line; copies source_outreach_id; one non-void invoice per authorization	Activity invoice.drafted	`CONFLICT` if one exists
saveInvoiceDraft	Shop user	line removals, lower quantities, lower prices only; mileage_out; expectedUpdatedAt. Any increase or new line → VALIDATION ("Use Add work to get the customer's approval")	Update draft; totals server-side	Activity invoice.edited (summary of changes)	`CONFLICT`

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
sendInvoice	Shop user	invoice_id, channel	Tx: assign number, freeze, public link (90 days, renewed on resend), message queued, reminders scheduled, PDF enqueued; record mileage_out as reading	Activity invoice.sent; audit	Total > 0; approved invoice_terms policy
resendDocument	Shop user	estimate or invoice id, channel	New message for the same frozen document; extends link expiry	Activity	Idempotent per clientRequestId
voidAndReissueInvoice	Shop user	invoice_id, reason	Tx: void (only if no succeeded payments); revoke link; cancel reminders; create new draft with same lines and `replaces_invoice_id`	Audit	Not allowed after any payment
createCheckoutSession	Public (token)	token	Verify invoice `sent`, balance > 0, shop charges_enabled; call Stripe (outside tx) to create Checkout Session on the connected account for the current balance with metadata (Section 11); store session ID on a `pending` payment row	Activity payment.started (first only)	Reuses an open unexpired session for the same balance; `PROVIDER_UNAVAILABLE` shows shop phone
recordManualPayment	Shop user	invoice_id, method (`cash`, `check`), amount = balance exactly, received_at, check_number	Tx: insert `succeeded` payment; invoice → `paid`; cancel reminders; enqueue receipt (if customer has contact) and attribution evaluation	Activity payment.recorded; audit	Amount ≠ balance → `VALIDATION`
voidManualPayment	Owner	payment_id, reason	Payment → `refunded` (void); invoice → `sent`; reverses attribution	Audit	Card payments not allowed (use Stripe)
applyStripeEvent	Worker	webhook_event id	Section 11 mapping; each event in its own tenant tx	Activity/audit per change	Idempotent by event status + unique Stripe IDs
startStripeOnboarding	Owner	—	Create Standard account if none (outside tx), store ID, create Account Link, return URL	Audit	`PROVIDER_UNAVAILABLE`

Messaging

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
submitTextingRegistration	Owner	business identity fields, EIN or sole-prop data, contact, attestations (checkboxes bound to policy version)	Store encrypted; state `submitted`; enqueue provider registration job	Audit texting.submitted	`VALIDATION` lists missing fields

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
sendReply	Shop user	conversation_id, body ≤ 1,600 chars	Eligibility check (registration approved, not opted out, shop number active); insert message queued (purpose `reply`); enqueue send	Activity message.sent	`NOT_ELIGIBLE` with reason
markConversationRead / archiveConversation	Shop user	id	Update	—	—
linkConversationCustomer	Shop user	conversation_id, customer_id or new customer fields	Sets customer phone if empty; never overwrites a different phone without confirm flag	Activity	`CONFLICT`
handleInboundSms	Worker (from Twilio webhook)	webhook_event	Resolve business by `To` number; upsert conversation; insert `received` message; keyword handling (STOP family → opt_out event; START/UNSTOP → opt_in_restored; HELP → no action, Twilio auto-reply configured); attach interaction to latest outreach to that number within 30 days	Activity message.received; consent audit	Idempotent on MessageSid
applyMessageStatus	Worker	webhook_event	Update message status if the new status ranks later (Section 19); error 21610 → opt_out event	—	Idempotent

Money Finder and attribution

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
getMoneyFinderSummary	Shop user	—	Read-only; Section 13 queries	—	—

Action	Caller	Inputs → validation	Transaction and side effects	Activity / audit	Idempotency / errors
listOpportunities	Shop user	category, cursor	Read-only; each item with eligibility per channel	—	—
dismissItem / snoozeItem / restoreItem	Shop user	category, subject_id, reason or days (30, 90)	Upsert recovery_item	Activity (customer)	—
previewOutreach	Shop user	category, subject_ids[], body	Renders per recipient, returns channel per recipient, segment count, send-window time, ineligible list	—	Body validation
sendRecoveryOutreach	Shop user	category, subject_ids[] (≤ 200), body, confirmed = true	Tx: re-check eligibility for each (Section 14); create batch, recovery_items (`contacted`), outreaches with tracking tokens, messages (`scheduled` for the window); recipients that fail eligibility are skipped and returned. After commit: sender job throttles	Activity per customer recovery.outreach_sent; audit batch	Idempotent on clientRequestId; `RATE_LIMITED` beyond daily cap
recordOutreachClick	Public (tracking token)	token	Insert interaction `link_click` , redirect to target public link	—	Idempotent per token per day
markResponse	Shop user	outreach_id, note	Interaction `owner_marked_response` (customer called or walked in)	Audit	Only within 30 days of outreach
reverseAttribution	Shop user	attribution_id, reason	Status `reversed`	Audit	—
createManualAttribution	Admin	outreach_id, invoice_id, kind, reason	Insert `is_manual` attribution after rule checks (same customer, invoice after outreach)	Audit	`CONFLICT` if invoice already attributed
evaluateAttribution	Worker	invoice_id	Section 12	Activity recovery.attributed	Deterministic; re-runnable

Internal admin: `createBusiness` , `startSupportSession` , `endSupportSession` , `importUpload` , `importDryRun` , `importCommit` , `importReverse` , `replayWebhook` , `approveLegalPolicy` (second admin), `runDataExport` . All admin-only, all audited with the internal user ID.

11. Provider integrations

Each provider sits behind one internal interface in `providers/` (`PaymentsProvider` , `MessagingProvider` , `EmailProvider` , `VinDecoder`). Domain code calls the interface; only the adapter knows provider names, IDs and error codes. Our database is the source of truth for documents, customers and attribution; providers are the source of truth only for what they own (whether money moved, whether a message was delivered, registration status).

Stripe

Account model: Stripe Connect, Stripe-managed pricing (D9): connected accounts with the shop paying Stripe's fees and using its own full Stripe dashboard (Standard-type). Confirm the exact account configuration with Stripe before Milestone 4 (Open Question 3).

Onboarding: `accounts.create` once per business → `account_links.create` (type onboarding, return and refresh URLs to S25) → status from `account.updated` webhooks plus an on-return `accounts.retrieve`. We store `charges_enabled`, `payouts_enabled`, `details_submitted`, `requirements.currently_due`.

Payment: direct charges on the connected account. When a customer taps Pay, create a Checkout Session (mode `payment`, card only, one line item "Invoice #1047 — {shop}", amount = current balance, currency USD, expires in 24 hours, success and cancel URLs back to P3/P4, `customer_email` if known). Pass `Stripe-Account` header. Idempotency key: `checkout:{invoice_id}:{balance_cents}:{date}`.

Metadata on session and payment intent: `business_id`, `invoice_id`, `invoice_number`, `public_link_id`, `outreach_id` (if the link came through a tracked outreach), `env`. Metadata is a hint for routing; we verify every value against our records (account ID must equal the invoice's business account).

Webhook events (Connect endpoint):

Event	Our action
`account.updated`	Update stripe_account; if charges disabled, hide Pay and alert owner
`checkout.session.completed`	If `payment_status = paid`: mark the pending payment `succeeded` with intent and charge IDs, update invoice, enqueue receipt and attribution
`checkout.session.expired`	Mark pending payment row `failed` (reason expired) silently; no customer impact
`payment_intent.succeeded`	Same as completed (whichever arrives first wins; the second is a no-op)
`payment_intent.payment_failed`	Record failed attempt on timeline

Event	Our action
`charge.refunded`	Upsert payment_refund rows from the charge's refunds; update payment status and invoice paid_cents; invoice may return to `sent`; reverse or reduce attribution
`charge.dispute.created` / `charge.dispute.closed`	Upsert payment_dispute; status changes; alert owner; on loss, treat as refund for attribution

Reconciliation: nightly job lists the last 3 days of charges per connected account and compares with payments; mismatches create alerts, never silent fixes.

Dashboard handoff: refunds, disputes evidence, payouts and receipts edits happen in the shop's Stripe dashboard. S13 shows "Refund in Stripe" linking to the payment in the dashboard.

Twilio

Architecture: ISV type #1 (D10). Our primary account holds our own brand for system messages (owner MFA codes, invites). Per shop: subaccount, Secondary Customer Profile, Brand (Standard; Low-Volume Standard by default since shop volume is small; Sole Proprietor path for shops without an EIN), Messaging Service, Campaign, one local number in the shop's area code.

Registration states map to Section 6: our state machine is driven by Twilio status of the profile, brand and campaign; `last_synced_at` updated by status callbacks and a poll every 6 hours while non-terminal. Rejection reasons are stored verbatim plus a plain-language mapping for common codes.

Campaign use case: one campaign per shop covering account notifications, customer care and service follow-ups, with sample messages for estimate delivery, invoice delivery, reminders and a recovery follow-up. Whether recovery outreach requires a marketing or mixed use case is Open Question 2.

Number provisioning: buy the number when the brand is approved (local area code; fall back to nearest), attach to the Messaging Service, status `provisioned`; becomes `active` and sendable only when the campaign is approved.

Inbound webhook: Messaging Service inbound URL → `/webhooks/twilio/sms`; signature validated (X-Twilio-Signature with the subaccount auth token); persisted to webhook_event; 200 returned immediately with empty TwiML; processed by worker.

Outbound: worker calls send with `MessagingServiceSid`, `To`, `Body`, `StatusCallback`. Before every send the worker re-runs eligibility (registration, consent, send window, cooldown for recovery). Bodies over 3 segments are rejected at compose time for recovery, allowed up to 10 for replies.

Delivery callbacks: `queued`, `sent`, `delivered`, `undelivered`, `failed` mapped to message status with rank ordering (Section 19).

STOP: Twilio's default opt-out handling stays on (it blocks sends and replies to STOP, START, HELP). We mirror every keyword into consent_event and treat error 21610 as an opt-out signal. We never rely on Twilio alone: our trigger blocks the send before it reaches Twilio.

Failure states: suspension or campaign deactivation → texting_account `suspended`, all queued SMS held, owner and support alerted, sends fall back to email where the purpose allows (estimate and invoice delivery, reminders), recovery batches pause.

Email fallback: while texting is not `approved` (or is `suspended`), estimate and invoice sends use email; if the customer has no email, the UI offers "Customer signs here" (estimate) or shows the link to copy (invoice) with a warning that copying links into personal texts is the owner's own message.

Email

Postmark, one server for transactional mail, sending domain ours with a per-shop display name ("Mike's Auto via [Product]") and Reply-To set to the shop's email. Shop-domain sending is NOT NOW.

Email	Trigger
Estimate sent / revised	sendEstimate (email channel)
Authorization receipt	submitAuthorization, if email on file
Invoice sent	sendInvoice (email channel)
Payment receipt	Payment succeeded
Invoice reminder, estimate follow-up	Scheduled sends when SMS not eligible

Email	Trigger
Recovery outreach (email channel)	Only when the customer has no eligible SMS path and email is eligible; includes one-click unsubscribe (P6) and List-Unsubscribe header
Owner notifications	Authorization received, payment received, texting state changes, provider failures
Login link, invite	Auth

Bounces and spam complaints arrive by webhook → email_suppression; suppressed addresses are skipped and the UI shows why.

VIN decoding

NHTSA vPIC `DecodeVinValues` behind `VinDecoder.decode(vin)`. Global cache keyed by VIN (180 days). 3-second timeout; one retry; failure returns "Couldn't decode — enter details manually." Decoded values pre-fill the form only; what the owner saves is the truth. A VIN that fails the check digit is accepted with a warning (shops encounter imports and older vehicles).

12. Recovered revenue engine

A dollar counts as directly recovered only when a stored chain links a specific outreach, a customer action on that outreach, the resulting work and a paid invoice. Everything else is at most influenced, shown separately, and never added to the headline.

The chain

1. **Opportunity** — a Money Finder item (recovery_item) with category and subject.
2. **Outreach** — one message to one customer about that item, with a unique tracking token.
3. **Interaction** — evidence the customer acted on that outreach: `reply` (inbound SMS from the same number within 30 days), `link_click` (tracking token), `estimate_approved_via_link`, `invoice_paid_via_link`, or `owner_marked_response` (customer called or walked in; owner taps "Customer responded" on the outreach; audited).

4. **Resulting work** — an estimate or appointment carrying `source_outreach_id` (set when created from the thread, the outreach row or the Money Finder item after an interaction), or the subject document itself (open estimate, unpaid invoice).
5. **Paid invoice** — `paid` status from succeeded payments.

Constants (stored in code with tests; changing them is a versioned rule change, not a setting)

- Interaction window: 30 days after the outreach is sent.
- Direct completion window: invoice paid within 60 days after the qualifying interaction.
- Influenced window: invoice sent within 45 days after the outreach was delivered.
- Amount: subtotal before tax of the counted invoice lines × (succeeded payments − refunds) ÷ invoice total, rounded to the cent. A fully paid invoice counts the full subtotal of its counted lines. Which lines count depends on the rule: opportunity-specific outreach (declined work, open estimate, unpaid invoice) counts only work tied to that opportunity; visit-generating outreach (lapsed, service due) counts the whole visit.

Direct rules

Rule	Applies to	Qualifies when	Amount
D-UNPAID	Unpaid invoice outreach	The subject invoice receives a succeeded payment after the outreach, either through the outreach's tracked link or within 14 days after any interaction on it	Pre-tax share of the payments made after the outreach
D-ESTIMATE	Open estimate outreach	The subject estimate is authorized (≥ 1 approved line) after an interaction (link approval is itself the interaction), and its invoice is paid within the completion window	Invoice subtotal

Rule	Applies to	Qualifies when	Amount
D-DECLINED	Declined work outreach	A new estimate with `source_outreach_id` = this outreach, or containing a line linked to the subject recommendation, is authorized after an interaction, and its invoice is paid within the window	Only the paid invoice lines that trace to the subject's recommendations (invoice line → estimate line → recommendation). Other work on the same visit is shown as "also on this visit, not counted"
D-VISIT	Lapsed and service-due outreach (and any category where the owner books from the thread)	An appointment or estimate with `source_outreach_id` is created within 30 days of an interaction, and an invoice from that estimate (or from an estimate linked to that appointment) is paid within the window	Invoice subtotal

Influenced rule (I-RETURN): the outreach was delivered (SMS `delivered` or email sent without bounce), the customer has a paid invoice first sent within the influenced window, and that invoice has no direct attribution. Only the first such invoice per outreach counts.

Conflicts and edge cases

- **One invoice, one attribution.** Enforced by the database (Section 8).
- **Direct beats influenced.** If a later evaluation finds a direct chain for an invoice holding an influenced record, the influenced record is reversed (reason `superseded_by_direct`) and a direct record created in the same transaction.
- **Several outreaches to one customer.** Direct goes to the outreach whose interaction is the latest one before the resulting estimate or appointment was created (last touch). Influenced goes to the most recent delivered outreach before the invoice was sent.
- **One outreach, several invoices.** Each invoice needs its own chain; two estimates created from the same reply can produce two direct records. Influenced is capped at one invoice per outreach.
- **Voided and reissued invoice.** The replacement inherits `source_outreach_id`; attribution follows the paid replacement.

- **Smaller work than offered.** Counted at what was invoiced and paid, never at the opportunity's original value. A $1,200 declined-work offer that becomes a $480 job counts $480.
- **Refunds and disputes.** Re-evaluated on every refund or dispute event; amount reduces; fully refunded or lost disputes reverse the record.
- **Voided manual payment.** Reverses the record.
- **Owner reversal.** Owner marks "Not from this outreach" with a reason; record `reversed`, audited, excluded from totals, still visible.
- **Manual addition.** Admin only, with reason, must still pass customer and timing checks; flagged `is_manual` and labeled as such in S6.
- **Determinism.** `evaluateAttribution(invoice_id)` runs on every payment, refund, dispute, void and reissue event, and a nightly job re-evaluates invoices paid in the last 120 days. Same inputs, same result. Any change writes an audit event with before and after.

Stored evidence: each attribution stores `rule_code`, ordered evidence references (outreach ID, message ID, interaction ID and kind, estimate and version IDs, authorization ID, invoice ID, payment IDs), the computed amount and inputs, and a generated explanation.

Examples

1. **Direct, declined work.** Mar 3: Dana declines front brakes ($612) on her 2017 Camry. May 10: owner sends declined-work outreach. May 10, 4:12 PM: Dana replies "how much again?" (interaction: reply). May 11: owner taps Re-quote in the thread → Estimate #2210 (source_outreach_id set): $640 of re-quoted brake work plus $202 for wipers and a cabin filter. May 12: Dana approves via link. May 14: Invoice #1047 paid, subtotal $842. Recorded: direct, D-DECLINED, $640 (only the brake lines that trace to her declined recommendation). Explanation: "Counted as directly recovered: Dana replied to your May 10 declined-work text, approved the re-quoted front brakes on Estimate #2210 on May 12, and paid Invoice #1047 on May 14. Counted: $640 of brake work. Also on this visit, not counted: $202."
2. **Influenced only.** Jun 1: lapsed outreach to Raul, delivered, no reply, no click. Jun 20: Raul walks in for an alternator; Invoice #1101 paid, $1,425. No interaction exists and the owner did not mark a response. Recorded: influenced, I-RETURN, $1,425. Shown under "Influenced (not counted as recovered)".

3. **Unpaid invoice.** Invoice #1032 ($300 total, $280 before tax) unpaid 21 days. Owner sends unpaid outreach; customer taps the tracked link and pays $300 by card. Recorded: direct, D-UNPAID, $280.
4. **Conflict.** Two outreaches to Lee: declined-work on Jul 1 (no response) and service-due on Jul 15 (reply). Lee books from the Jul 15 thread and pays Invoice #1150. Direct goes to the Jul 15 outreach; the Jul 1 outreach gets nothing (an invoice holds one attribution).

What the owner sees: S1 card "Recovered this month: $X directly" with "plus $Y influenced" in smaller type on its own line. S5 lists each record with its explanation. There is no screen, export or email that adds the two numbers.

13. Money Finder query rules

Five deterministic SQL queries, no AI, each with a fixture-based test suite. Lists show every matching item; contact eligibility (Section 14) is shown per item, not used to hide items.

Exclusions shared by all lists: customer archived, deleted or `do_not_contact`; item dismissed; item snoozed until a future date; vehicle `sold` or `inactive` (vehicle-based items only).

List	Required data	Eligible when	Extra exclusions	Dollar value	Ordering	Grouping and dedup	Resolves when
Declined work	recommendation rows	status `open`, declined within the last 18 months	Recommendation `quoted` (a re-quote is out)	Sum of open recommendation amounts (pre-tax line totals) per customer and vehicle	Value desc, then most recent decline	One item per customer + vehicle, listing each recommendation	All grouped recommendations are `won`, `lost` or `dismissed`
Open estimates	estimate + current sent version	Status `sent` or `viewed`; sent ≥ 2 days ago; unexpired, or expired within the last 60 days (shown with "Renew first")	Void, authorized, declined	Version subtotal	Value desc	One item per estimate	Authorized, declined, voided, or 60 days past expiry. Expired items can't be included in outreach until renewed (revise + send, two existing actions)
Unpaid invoices	invoice	Status `sent`, balance > 0, sent ≥ 3 days ago	Open dispute	Balance due (what the customer owes, tax included)	Age desc (oldest first)	One item per invoice	Paid or voided

List	Required data	Eligible when	Extra exclusions	Dollar value	Ordering	Grouping and dedup	Resolves when
Not back in 6+ months	paid invoices and historical_sale	Most recent visit (paid invoice date or historical sale date) between 6 and 24 months ago	Customer appears in declined work, open estimates or unpaid invoices (those lists take priority); future appointment exists; lapsed outreach sent in last 90 days	Shown per customer as "average visit $X" = mean of the last ≤ 5 visit amounts. Card total labeled "est." and excluded from the known-dollars banner	Lifetime spend desc	One item per customer	A new paid invoice, an appointment, or 24 months passes
Service due	asset.next_service_on (owner-entered)	Due date between 90 days ago and 14 days ahead	Future appointment for that vehicle; a paid invoice for that vehicle dated within 30 days before the due date or later	`next_service_amount_cents` if entered; otherwise "value not set"	Due date asc	One item per vehicle	Owner updates or clears next service, or a qualifying paid invoice

Overview banner (S1): "$X in money you can go after" = declined work + open estimates + unpaid invoices + service due items that have an entered amount. Lapsed customers appear as a count with an estimated value on their own card. Known and estimated dollars are never summed.

Dismiss reasons: customer moved, sold vehicle, work done elsewhere, don't want to contact, other (free text). Dismissing a declined-work item marks its recommendations `dismissed`; restoring returns them to `open`.

Snooze: 30 or 90 days; item returns automatically when the snooze ends.

Performance: each query bounded by business and indexed (Section 7); target < 300 ms at 20,000 customers. Computed live on request; no stale snapshot tables in MVP.

14. Messaging and compliance model

The platform, not the owner, enforces consent, STOP, send windows and volume limits. Where this section depends on law, it says so and defers to counsel; those points are marked **[COUNSEL]**.

Message purposes

Purpose	Examples	Who starts it	Consent basis needed (default until counsel rules)
`transactional`	Estimate delivery, invoice delivery, payment receipt, authorization receipt	Owner action on a document the customer asked for	Phone provided by the customer to the shop for this service, and not opted out **[COUNSEL]**
`reminder`	Estimate follow-up, invoice reminder	System, tied to that document	Same as transactional **[COUNSEL]**
`reply`	Owner's message in a conversation	Owner	Customer texted the shop, or transactional basis; not opted out
`recovery`	Money Finder outreach	Owner selection and send	**Recovery consent recorded** for that number (see below) **[COUNSEL]**
`system`	Invites, login codes	Platform	Not sent to the shop's customers

Consent state per phone number (consent_state, derived from the consent_event ledger):

- `transactional_allowed`: true unless the number has opted out.
- `recovery_allowed`: true only with a recovery-scope `opt_in` event from an accepted source and no later opt-out.
- Accepted recovery sources by default: `authorization_checkbox` (unchecked by default on P1; text from the `sms_consent_disclosure` policy) and `owner_recorded` (owner attests how the customer agreed, with a note). `import_attested` (the shop's attestation at import that customers gave their number for service contact) is stored for every imported customer but enables recovery texting **only if counsel approves it for that jurisdiction**, via a flag on the Texas policy. The same policy can list recovery categories (for example unpaid invoices) that counsel allows on transactional consent.

- **Business impact, stated plainly:** under the conservative default, most imported lapsed customers can be reached by email but not by text until they opt in. This is the most important unresolved decision (Open Question 1).

Email recovery: allowed to customers with an email who have not unsubscribed and are not suppressed; every recovery email carries a one-click unsubscribe and the shop's postal address **[COUNSEL]**.

Keywords: Twilio's opt-out handling stays on. STOP family (STOP, STOPALL, UNSUBSCRIBE, CANCEL, END, QUIT, OPTOUT, REVOKE) → `opt_out` scope `all` for that shop's number; START family (START, UNSTOP, YES) → `opt_in_restored` (transactional only; recovery consent is not restored by START); HELP/INFO → Twilio's configured help reply with the shop name and opt-out instructions. Opt-out applies per shop, not across shops.

Send windows: recovery and reminder messages send only Monday–Saturday 9:00 AM–8:00 PM and Sunday noon–8:00 PM in the shop's timezone; outside the window they queue for the next opening. Transactional sends and replies the owner initiates go immediately. Exact Texas hours and holiday rules **[COUNSEL]**; values live in the jurisdiction policy, not in code.

Templates: we ship defaults for every kind, with counsel-reviewed recovery wording. Recovery and reminder SMS always begin with the shop name (system prefix "Mike's Auto: ", not editable) and end with the `recovery_footer` policy text (for example the opt-out instruction), not editable. Owner-edited bodies: maximum 320 characters for recovery (two segments), no URLs except our tracked `{link}`, no merge fields outside the allowed list. The final rendered body is stored on each message.

Bulk-send safeguards

- Maximum 200 recipients per batch; 300 recovery messages per shop per day.
- Per customer: no recovery message within 14 days of the last one; maximum 3 recovery messages in any 90 days.
- A shop's first batch over 50 recipients is held for support review (checks template and data quality) before sending.
- Throughput: one message per second per shop number by default, adjusted to the campaign's carrier limit.
- Auto-pause recovery for a shop when a batch's opt-out rate exceeds 5% or failure rate exceeds 15%; owner and support alerted; support resumes.

- Global kill switches: recovery sending per shop and platform-wide, operable from admin.

Audit trail: batch audit event (who, template as edited, recipient count, channel counts); every message row with final body, purpose, consent snapshot reference and provider ID; consent ledger append-only; kill-switch changes audited.

Requires legal/compliance verification before the first real send [COUNSEL]: consent basis for transactional texts and reminders; whether recovery outreach is marketing under TCPA and Texas law, and what consent it needs; whether import attestation can ever qualify; quiet hours; required identification and opt-out text; Twilio campaign use-case choice; record retention for messages and consent evidence.

15. Import specification

Our team runs every import from the admin console (I2). The pipeline is staged, previewed, idempotent and reversible; nothing touches live tables until an admin commits a reviewed dry run.

Source profiles. Export column names vary by product version, so each profile is a header-alias map plus parsing rules, editable by admins in I2 as JSON. That internal map is not a customer-facing mapping wizard.

Profile	Files we ask the shop for	Used for
Square	Customer directory export; Transactions export (and Items detail if available) for the full history range	Customers (name, phone, email, Square customer ID); sales (date, net amount, transaction ID, customer ID, item names for the description)
QuickBooks Online / Desktop	Customer contact list; Sales by Customer Detail report (or Transaction List by Customer) for the full range	Customers; sales (date, transaction type, number, customer, amount, memo)

Profile	Files we ask the shop for	Used for
Generic CSV	Our three templates: `customers.csv` (external_id, first_name, last_name, phone, email, notes), `vehicles.csv` (customer_external_id, vin, year, make, model, plate, mileage), `sales.csv` (sale_id, customer_external_id or phone, date, amount, description)	Old shop systems and spreadsheets, reshaped by us

Pipeline

1. **Upload** — files stored privately as `import_source`; job `uploaded`.
2. **Parse** — CSV with encoding detection (UTF-8, Windows-1252); XLSX converted; each row stored raw in `import_row`.
3. **Normalize** — names trimmed and case-fixed only when all-caps; phone to E.164 (US default) or flagged; email lowercased; dates parsed with the profile's explicit format in the business timezone; amounts to cents (handles `$`, commas, parentheses as negative); VIN uppercased and validated; plate uppercased.
4. **Validate** — each row `valid`, `warning` (imported with a note) or `error` (not imported). Errors: no name and no phone and no email; unparseable date or amount on a sale. Warnings: invalid phone (customer imported without phone), VIN check-digit failure, duplicate in file.
5. **Match customers** — within the file by external ID; then against existing customers by exact phone, then exact email. Name-only similarity never merges; it produces a warning for review. Matched existing customers are only filled where fields are empty, never overwritten.
6. **Match vehicles** — by VIN; else by customer + year + make + model.
7. **Sales** — become `historical_sale` rows linked through the customer mapping. Zero amounts skipped. Negative rows (refunds) net against the same transaction ID when present, otherwise skipped and reported. Voided transactions skipped.
8. **Dry run** — job `dry_run_ready` with a report.
9. **Commit** — admin confirms the owner's attestation (checkbox with owner name, date and the policy text shown during the call), then commits in transactions of 500 rows. Every created or filled record carries `import_job_id`; `customer.last_visit_at` recomputed; a consent event `import_attested` written per imported phone.

10. **Report** — committed counts stored on the job and emailed to support; error rows downloadable as CSV.

Dry run report contents: customers new / matched / skipped; vehicles new / matched; sales imported, total dollars and date range; rows with errors and warnings by reason; customers with no phone and no email (imported but unreachable); a preview of the Money Finder "Not back in 6+ months" count and estimated value; 20 sample normalized rows. The admin reviews it on screen-share with the owner before commit.

Idempotency: each row gets `source_key` = hash(profile, entity, external ID — or the normalized identifying fields when no ID exists). A committed source key is unique per business, so re-running the same file creates nothing new. Historical sales are unique on `(source, source_ref)`.

Reversal: allowed within 30 days of commit. Deletes historical sales from the job; deletes customers and vehicles the job created only if untouched (no estimates, invoices, messages, notes, appointments, outreach); restores filled fields on matched customers from the stored before-values. Untouched records reverse; touched ones are kept and listed in the reversal report. Audited.

Limits and hygiene: 100,000 rows per job; one active job per business; raw rows purged 90 days after commit; import files deleted after 90 days.

16. Public customer pages

No-login pages that expose exactly one document and capture authorization evidence a court could read. Routes: `/e/{token}` (estimate), `/i/{token}` (invoice), `/r/{token}` (outreach tracking redirect).

Token security: 32 random bytes, base64url (43 characters); only the SHA-256 hash is stored; looked up by the SECURITY DEFINER resolver (Section 9); tokens never appear in logs (route logging masks the path segment).

Expiration and revocation

Link	Valid	Revoked when
Estimate	Until estimate expiry + 7 days; approval disabled after expiry (view only)	A newer version is sent, the estimate is voided

Link	Valid	Revoked when
In-shop signing	15 minutes, single authorization	Used or expired
Invoice	90 days from the most recent send; once paid, receipt view for 1 year	Invoice voided
Outreach tracking	30 days; redirects to the current link of its subject	Subject resolved: redirects to a "no action needed" page with the shop phone

Rate limits: 60 requests per minute per IP across public routes; 20 failed token lookups per IP per 10 minutes → 429; authorization submit 5 attempts per link per 10 minutes; checkout creation 10 per invoice per hour.

Data exposed

- Estimate: shop name, logo, address, phone; customer full name; vehicle year/make/model and last 6 of VIN; lines, photos (10-minute signed URLs, only this version's), customer note, totals, expiry date, authorization text from the approved policy.
- Invoice: shop header; customer name; vehicle; lines and totals; status, amount paid, payment method type (card, cash, check) with no card details; Pay button only if balance > 0 and shop charges are enabled.
- Never: other documents, customer phone or email, notes, timeline, mileage history, internal user names.

Response headers: `Cache-Control: no-store`, `X-Robots-Tag: noindex`, `Referrer-Policy: no-referrer`, strict CSP, frame-ancestors none.

Mobile layout: single column from 320 px, 16 px minimum text, approve/decline toggles ≥ 44 px, sticky bar with approved total and Submit, signature pad full width and 200 px tall with Clear, photos open full screen with pinch zoom. Requires JavaScript for signing; without it the page shows the estimate and the shop phone.

Authorization evidence captured server-side: version ID and its snapshot hash; every line decision; approved and declined totals; typed signer name; signature PNG; server timestamp; client IP (first hop as reported by the hosting platform's trusted header); user agent; link ID and method (`online` or `in_person` with staff user); policy version and SHA-256 of the exact authorization text displayed; consent checkbox state. A PDF of all of it is rendered and stored, and the customer receives a copy by email when one is on file.

17. Files and photos

Files are private, immutable objects addressed by ID; a sent document references exact file IDs, so what the customer saw can always be reproduced.

- **Upload:** browser asks the server for an upload slot (`createUpload` : kind, content type, byte size) → server creates a `pending` file row and a signed upload URL (5 minutes) → browser uploads directly to storage → `completeUpload` enqueues processing.
- **Validation:** JPEG, PNG, WebP, HEIC/HEIF only, checked by magic bytes, not extension; maximum 15 MB input; maximum 10 photos per estimate version. Anything else → `quarantined` and deleted.
- **Processing job:** auto-orient, convert HEIC to JPEG, strip all metadata including GPS, resize to 2,048 px on the long edge, quality 80, plus a 400 px thumbnail; compute SHA-256; status `ready` . Re-encoding also neutralizes malformed image payloads.
- **Key strategy:** `b/{business_id}/{kind}/{yyyy}/{mm}/{file_id}.{ext}` . No names, phone numbers or VINs in keys.
- **Tenant isolation:** private bucket, no public URLs; the server issues signed read URLs (5 minutes in the shop, 10 minutes on public pages) only after a tenant-scoped lookup proves the file belongs to a record the viewer may see.
- **Signatures:** PNG ⩽ 200 KB, written by the server during `submitAuthorization` , kind `signature` .
- **PDFs:** estimate, invoice and authorization PDFs rendered by the worker from the frozen data and stored once per sent version or invoice; re-rendering is never used to replace a stored PDF.
- **Immutability and snapshots:** files are never overwritten. Revising an estimate copies photo references (not bytes) to the new draft; removing a photo from a draft does not affect sent versions.
- **Deletion and retention:** unreferenced `pending` or draft-only files deleted after 24 hours by the cleanup job; files referenced by sent versions, authorizations or invoices retained for the life of the account and per the retention policy **[COUNSEL]**.

18. Background jobs

Graphile Worker on Postgres: jobs are enqueued in the same transaction as the data that caused them, so a committed change always has its job and a rolled-back change never does. Every job runs inside `withTenant` when it touches tenant data.

Job	Trigger	Retry policy	Idempotency
process_webhook_event	Each stored webhook	8 attempts, exponential 1 min → ~4 h, then `dead` + alert	Event status with row lock; unique provider IDs on effects
send_sms	Message row created or released from schedule	Transient errors 5 attempts (30 s → 30 min); permanent errors none	Row lock, status must be `queued`; on timeout with unknown outcome, status `sending` is reconciled by querying the provider for messages to that number in the last 10 minutes with the same body hash before any resend
send_email	Same, for email	5 attempts	Status check + message ID in Postmark metadata
run_scheduled_sends	Cron every 5 minutes	Next run retries	`FOR UPDATE SKIP LOCKED` on due rows; re-checks target state; unique message idempotency key per step

Job	Trigger	Retry policy	Idempotency
release_window_messages	Cron every 5 minutes	Next run	Releases `scheduled` messages whose time and send window are open
send_recovery_batch	After sendRecoveryOutreach commit	Per message as send_sms; batch resumes where it stopped	Each outreach has one message with key `{outreach_id}`
render_pdf	Document sent; authorization recorded	5 attempts, then alert; document link still works without the PDF	Skip if `pdf_file_id` already set
process_upload	completeUpload	3 attempts, then `quarantined`	Skip if `ready`
vin_decode	Vehicle saved with VIN and empty fields	2 attempts; then leave as is	Only fills empty fields
import_parse, import_dry_run, import_commit, import_reverse	Admin action	No automatic retry; admin re-runs	Source keys and job status
evaluate_attribution	Payment, refund, dispute, void, reissue events	5 attempts	Deterministic result; one active attribution per invoice

Job	Trigger	Retry policy	Idempotency
attribution_sweep	Nightly 02:00 CT	Next night	Re-evaluates invoices paid in last 120 days
expire_estimates	Hourly	Next run	Transition only from `sent` / `viewed` past `expires_at`
stripe_account_sync	Onboarding return; daily	3 attempts	Overwrites status fields from Stripe
stripe_reconcile	Nightly	Next night	Read-only compare; alerts on mismatch
texting_registration_sync	Every 6 hours while non-terminal; on callbacks	Next run	Overwrites provider status fields; transitions validated
cleanup_files	Daily	Next run	Deletes only unreferenced files older than 24 h
purge_import_rows	Daily	Next run	Rows of jobs committed > 90 days
purge_request_idempotency	Daily	Next run	Rows > 48 h
data_export	Admin action	3 attempts	One export file per request
heartbeat	Every minute	n/a	Writes worker heartbeat for queue-lag alerting

19. Failure modes

Rule: the owner is never told money moved or a message went out until the provider confirms it, and any asynchronous failure that could cost money becomes a visible alert in the app and to support.

Failure	Behavior
Stripe unavailable	Pay button returns "Card payments are temporarily unavailable — call the shop" with the shop phone; nothing recorded. Onboarding shows retry. Cash/check unaffected
Twilio unavailable	Messages stay `queued` and retry; after 30 minutes undelivered, estimate and invoice sends alert the owner with "Send by email instead"; recovery batches pause and resume automatically
Email (Postmark) unavailable	Emails queue and retry for 6 hours; then owner alert listing affected documents with copyable links
VIN provider unavailable	Decode returns quickly with "enter details manually"; nothing blocks saving
Database transaction conflict	Serialization or deadlock errors retried up to 3 times inside the action wrapper; then `CONFLICT` to the user with their input preserved
Worker delayed or down	Heartbeat alert after 3 minutes; queue age alert after 10 minutes; web app continues; owner sees "Sending…" not "Sent"
Webhook duplicated	Unique provider event ID: second copy stored as duplicate and ignored
Webhook out of order	Status updates use rank order (`queued` < `sent` < `delivered` / `undelivered` / `failed`); a lower rank never overwrites a higher one, except a late `undelivered` after `delivered` , which is recorded as a note. Stripe objects are re-fetched from Stripe before applying when the event's object is older than our stored state

Failure	Behavior
SMS number or campaign suspended	Texting state `suspended`; queued SMS held; estimate/invoice sends switch to email; recovery paused; owner and support alerted with provider reason
Payment succeeded, webhook delayed	P4 shows "Payment processing — you'll get a receipt shortly" (never "paid"); on return, the page asks the server to retrieve the Checkout Session from Stripe and applies it through the same idempotent path; nightly reconciliation catches anything left
Two card payments for one balance (race)	Both recorded (money moved); invoice marked overpaid; owner and support alerted to refund the extra in Stripe
Import partially fails	Commit is batched; a failed batch stops the job (`failed`) with committed batches intact and reported; re-running skips committed source keys
PDF generation fails	Document still sendable and viewable as a web page; PDF shows "preparing"; after 5 failed attempts support is alerted
Attribution job fails	Retries; nightly sweep recomputes; report shows "last updated" time
Stripe account disabled	Pay hidden, owner alerted with Stripe's requirement text and a link to resolve
Storage unavailable	Photo upload shows retry; documents already sent keep working when storage returns; sending a new estimate with unprocessed photos is blocked until photos are `ready`

20. Observability

Every log line, job and provider call carries `request_id` and `business_id`, so support can trace one shop's problem end to end without reading customer data.

Structured logs (JSON, one line per event): timestamp, level, request_id, business_id, actor_type, actor_id, action or job name, provider and provider object IDs, duration, outcome, error code. **Never logged:** message bodies, customer names, full phone numbers (last 4 only), emails (masked), tokens, signature data, EIN, card data, raw webhook payloads (stored in webhook_event, not logs).

Tracing: request ID generated at the edge, passed into server actions, copied into job payloads and provider metadata where allowed; Sentry performance tracing for web requests and jobs.

Metrics: action latency and error rate; queue depth and oldest-job age per job type; job failures; webhooks received, applied, failed, dead per provider; messages by status per purpose; payment success and failure counts; texting registrations by state; Money Finder query time; public-link 404/429 rates.

Alerts (to the on-call engineer by SMS and email):

Alert	Threshold
Web error rate	> 2% of requests over 5 minutes
Worker heartbeat missing	3 minutes
Oldest queued job	> 10 minutes
Webhook `dead`	Any
Payment reconciliation mismatch	Any
Overpayment recorded	Any
SMS failure rate per shop	> 15% in an hour
Texting account suspended	Any
Recovery auto-pause triggered	Any
RLS test canary fails in production	Any (a scheduled job queries as tenant A for a seeded tenant B row and expects zero)
Backup job failure	Any
Stripe account disabled for a live shop	Any

21. Analytics and activation

Product analytics answer one question: is the Money Finder producing paid work? Events are sent server-side to PostHog with `business_id` as the group and no customer PII.

Event	Fired when	Key properties
import_completed	Import committed	source, customers_imported, sales_imported, sales_total_cents, date_range
customer_created	Customer saved	source
estimate_created	First draft save	line_count
estimate_sent	sendEstimate	channel, total_cents, version_no
estimate_viewed	First public view	minutes_since_sent
estimate_line_approved / estimate_line_declined	Authorization recorded (one per line)	line_type, amount_cents
authorization_recorded	Authorization	method, approved_cents, declined_cents
invoice_sent	sendInvoice	channel, total_cents
payment_completed	Payment succeeded	method, amount_cents, days_since_sent
money_finder_opened	S1 viewed	known_value_cents
recovery_list_viewed	S2 viewed	category, item_count, value_cents
recovery_outreach_sent	Batch sent	category, recipients, sms_count, email_count, edited_template (bool)
recovery_reply_received	Reply interaction	category, hours_since_sent
recovery_link_clicked	Click interaction	category
directly_recovered_revenue	Direct attribution created	category, rule_code, amount_cents

Event	Fired when	Key properties
influenced_revenue	Influenced attribution created	category, amount_cents
attribution_reversed	Reversal	kind, by (owner/admin)
texting_state_changed	Registration state change	from, to
stripe_charges_enabled	Charges enabled	—

Activation (design partner is "live"): within 14 days of import commit, the shop has (1) sent at least one estimate or invoice through the product and (2) sent at least one recovery outreach batch to 10 or more customers.

Value moment: the shop's first directly recovered revenue record. Time from import to value moment is the headline metric we review weekly with each design partner.

22. Testing strategy

Tests run against real Postgres with the production schema, roles, RLS and triggers; nothing that guards money, tenants or consent is tested with a mocked database.

Layer	What	Tooling
Unit	Money math (tax rounding half-up, line totals, subtotal share), phone and email normalization, template rendering and validation, send-window calculation, every state machine (all legal transitions pass, all listed invalid ones fail)	Vitest
Domain	Server actions end to end inside a transaction against a test database: estimate send freezes version, authorization creates recommendations, invoice paid status, consent effects, cooldowns	Vitest + local Supabase Postgres, fresh schema per run

Layer	What	Tooling
Database constraints	One test per Section 8 invariant that attempts the violation directly in SQL as `app_user` and expects a specific error	Vitest + SQL
RLS	Generated for every tenant table: insert in A, select as B → 0 rows; insert as B with A's IDs → error; CI schema check that every `business_id` table has RLS enabled, forced and a policy	Vitest + catalog queries
Cross-tenant	Section 9 matrix through server actions, public routes, signed URL issuance, worker jobs and webhooks	Vitest
Provider adapters	Contract tests against recorded fixtures; Stripe test mode and Stripe CLI event fixtures; Twilio signature validation with fixture payloads; Postmark test token	Vitest
Webhooks	Invalid signature rejected; duplicate ignored; out-of-order status ranks; unknown event types stored as `ignored`; replay produces no double effects	Vitest
Public links	Expired, revoked, superseded, unknown token (identical response); rate limits; response field allowlist snapshot (fails if any non-listed field appears)	Vitest + Playwright
Money Finder	Fixture business with known data; each list's eligibility, exclusions, values, ordering, dedup	Vitest
Attribution	The four Section 12 examples plus refunds, voids, conflicts, reversal, manual add, sweep determinism	Vitest

Layer	What	Tooling
Import	Square, QuickBooks and CSV fixture files: dry run counts, idempotent re-run, reversal	Vitest
End to end	Critical flows at phone (390 × 844) and desktop viewports	Playwright on preview/staging
Performance sanity	Money Finder queries and search at 20,000 customers / 60,000 sales	Seeded benchmark in CI weekly

Deployment-blocking E2E scenarios

1. Sign in by email link; Staff cannot reach Owner-only actions.
2. Create customer and vehicle with VIN decode (stubbed provider).
3. Estimate with photos → send by email → public page approve two lines, decline one → sign → authorization evidence complete → recommendation created.
4. Invoice from estimate → send → Stripe test card payment → webhook → invoice paid → receipt sent.
5. Record cash payment → invoice paid.
6. Texting not approved → estimate send is forced to email with explanation.
7. Texting approved (staging number, allowlisted phone) → outbound and inbound SMS in the thread with delivery status.
8. STOP from the customer → consent opted out → reply composer disabled → queued reminder cancelled.
9. Money Finder declined work → outreach → reply → re-quote → approve via link → pay → direct attribution with correct explanation; separate customer produces influenced only.
10. Cross-tenant: user of A opening B's customer, estimate and invoice URLs gets not-found.
11. Superseded and expired estimate links cannot be approved.
12. Import fixture: dry run → commit → Money Finder lapsed list populated → reversal removes untouched records.

23. Development environments

Four environments with separate databases, keys and storage. No environment can send a real text or email to a non-team recipient except production.

Environment	Database	Stripe	Messaging	Email	Data
Local	Supabase CLI local stack (Postgres, Auth, Storage) in Docker	Test mode; Stripe CLI forwards webhooks	`fake` adapter: messages appear in a dev-only inbox page; inbound simulated from that page	Mailpit (local catcher)	Seed script: tenants A and B, 2,000 customers, vehicles, 18 months of sales, estimates and invoices in every state
Preview (per pull request)	Supabase branch database, migrations applied, seeded	Test mode	`fake` adapter	Postmark test token (no delivery)	Seeded
Staging	Persistent Supabase project	Test mode with a test connected account	Real Twilio subaccount with a registered test brand and campaign; adapter refuses any recipient not on the team allowlist	Postmark sandbox server; allowlisted recipients only	Seeded tenants plus a copy-of-production-shape synthetic dataset; never real customer data
Production	Separate Supabase project, PITR on	Live platform	Live, per-shop subaccounts	Live server	Real

Secrets: stored in Vercel and Render environment settings per environment; `.env.example` lists names only; nothing secret in the repository. Per-shop Twilio subaccount auth tokens and shop EINs are encrypted at rest in the database with a key held in environment settings (envelope encryption with pgcrypto). Keys are rotated when anyone with access leaves.

Test data: Stripe test card numbers; Twilio test credentials and magic numbers for adapter tests; staging's allowlisted phones belong to the team.

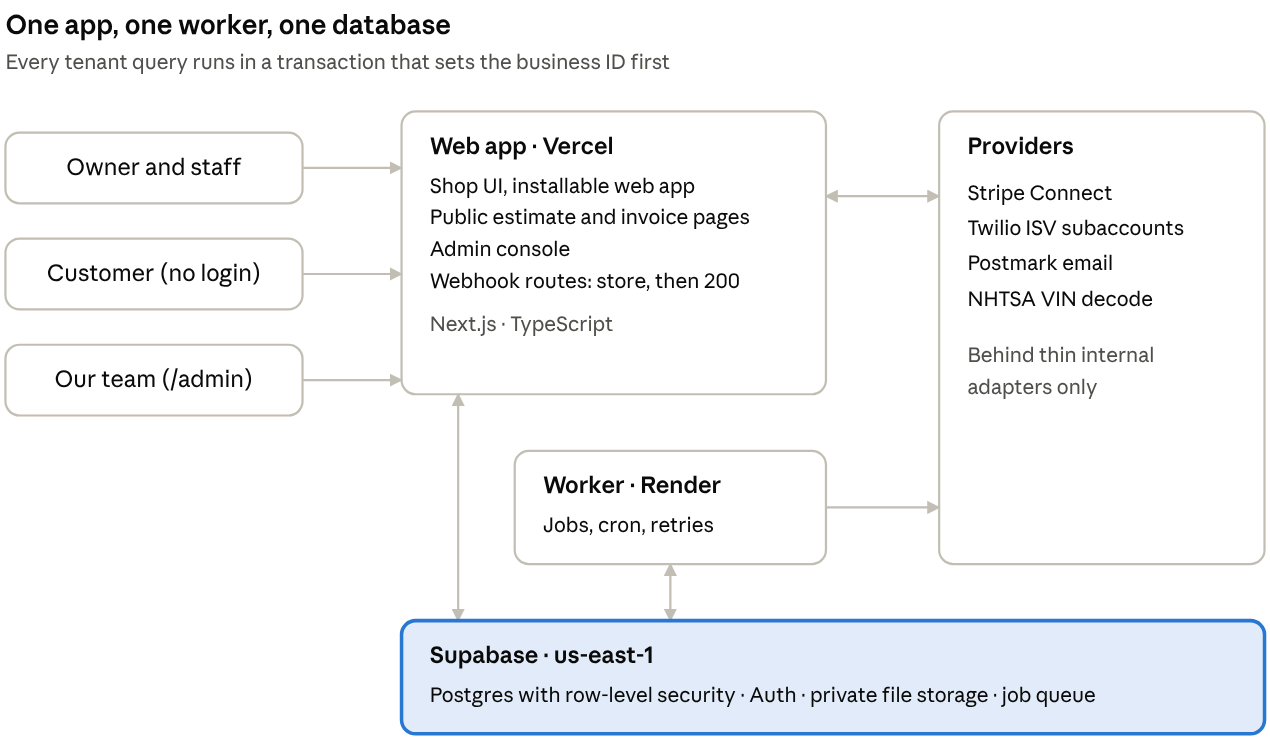
24. Migration strategy

Migrations are reviewed SQL files in Git from the first commit, applied forward-only, and always compatible with the app version already running.

- **Versioned:** Drizzle Kit generates table changes; RLS policies, triggers, grants, partial indexes and SECURITY DEFINER functions are hand-written SQL in the same migration sequence. Every migration file is reviewed as SQL in the pull request.
- **Forward only in production:** no down migrations. A bad migration is fixed by a new forward migration; data loss is recovered from PITR.
- **Expand, then contract:** breaking changes ship in steps across deploys: add nullable column or new table → deploy code writing both → backfill → deploy code reading new → enforce NOT NULL or constraint → drop old in a later release.
- **Data migrations:** separate idempotent scripts run as worker jobs in resumable batches (1,000 rows per transaction), never inside the schema migration for large tables.
- **Production safety:** `lock_timeout = 5s` and `statement_timeout = 60s` for migrations; indexes on existing large tables created `CONCURRENTLY`; CI lints for dangerous operations; CI applies the full chain to an empty database and to a staging snapshot; the RLS presence check runs after migration; deploy verifies a PITR restore point exists before applying production migrations.
- **Order:** CI → migrate staging → deploy staging → E2E → approve → migrate production as `migration_owner` → deploy app. Because of expand/contract, old app code keeps working during the minutes between migrate and deploy.

25. Deployment architecture

One web app, one worker, one managed Postgres, plus providers. All in US East to keep database round-trips short.



MVP deployment topology

The web app and the worker are the only code we run. The app calls Stripe and VIN decoding directly for interactive steps and receives every webhook; the worker makes all other provider calls. Both reach the database only through tenant-scoped transactions.

Component	Service	Notes
Web app	Vercel (Next.js): shop UI, public pages, `/admin`, webhook routes	Region `iad1`; connects through the Supabase transaction pooler
Short link domain	Same Vercel app on a short domain for `/e`, `/i`, `/r`	Keeps SMS short; same code and security
Worker	Render background worker (Node, Graphile Worker + crontab), 1 instance; 2 before shop #5	Direct (session) Postgres connection for LISTEN/NOTIFY
Database	Supabase Postgres (paid tier), PITR 7 days	Region `us-east-1`
Auth	Supabase Auth	Email link and SMS code

Component	Service	Notes
Files	Supabase Storage, private buckets `files` and `imports`	Signed URLs only
Payments	Stripe Connect	Webhook endpoint on web app
SMS	Twilio, ISV subaccounts	Webhooks on web app
Email	Postmark	Bounce webhooks on web app
VIN	NHTSA vPIC	No key
Errors and tracing	Sentry	Web and worker
Logs and uptime	Better Stack	Log drains from Vercel and Render; uptime checks on public pages and the health route
Analytics	PostHog (US cloud)	Server-side events
Off-site backups	Nightly `pg_dump` and storage sync by a scheduled GitHub Action to an encrypted S3 bucket in a separate AWS account, 35-day retention	Restore drill before the first shop and quarterly after
CI/CD	GitHub Actions	Tests, migration checks, preview deploys, gated production deploy

Not used: Kubernetes, microservices, Redis, a message broker, a search cluster, a separate API service. Each would add operations without serving the first 10 shops.

26. Implementation milestones

Nine vertical slices, each shippable to staging and demoable on a phone. With two engineers working two tracks, the realistic calendar is 12–14 weeks, not the strategy's 10–12; the strategy's number assumed less hardening than this spec requires.

Two tracks after Milestone 1: Track A builds the document and money path (M2 → M3 → M4 → M8). Track B builds the provider- and data-heavy pieces (M5 after M1, M6 after M2, M7 after M3 and M6). M9 needs both.

Start these non-code tasks in week 1 because they have external lead times: our Twilio ISV primary profile and brand; Stripe Connect platform setup and confirmation of the account configuration; engaging Texas counsel for Open Questions 1, 2, 4 and the legal policies; choosing the first 10 shops and collecting their exports.

M1. Foundation and tenant isolation (≈ 2 weeks)

- **Delivers:** repository, CI, local/preview/staging/production environments; schema for business, location, user, membership, internal_user, support_session, activity, audit_event, request_idempotency, webhook_event, legal_policy, file; roles and RLS; `withTenant`; sign-in, invite and Staff slot; admin sign-in, tenant list, view-as, legal policy manager; generic webhook inbox with replay; Sentry, logs, health route; off-site backup job.
- **Acceptance:** two seeded tenants; full RLS and cross-tenant suites green; CI fails a migration that adds a tenant table without RLS; Owner invites Staff; Staff blocked from Owner-only actions; view-as is read-only and appears in the support access log; production deploy pipeline runs migrations; one backup restored successfully into a scratch project.
- **Dependencies:** none.
- **Risks:** RLS with pooled connections (mitigated by `SET LOCAL` in every transaction and a test that runs 1,000 interleaved requests across tenants); Supabase Auth to membership mapping.
- **Tests:** RLS generator, isolation matrix (identity parts), auth E2E #1, constraint tests for membership rules.

M2. Customer and vehicle memory (≈ 1.5 weeks)

- **Delivers:** customers, vehicles (asset + vehicle), VIN decode with cache, mileage readings, notes, timeline, global search, Today/Tomorrow and appointment sheet, the mobile shell (tabs, New sheet, settings menu), business profile and onboarding S21/S22. The Money tab shows only the setup checklist until M7; no placeholder numbers.
- **Acceptance:** E2E #2; search finds a customer by any phone format, plate, VIN last 6; timeline shows customer, vehicle, note and appointment events; all screens usable at 390 px.
- **Dependencies:** M1.
- **Risks:** search performance at 20,000 customers.

- **Tests:** normalization unit tests, search fixtures, cross-tenant for every new action.

M3. Estimate to customer authorization (≈ 2.5 weeks)

- **Delivers:** estimates, versions, lines, saved lines, photos (upload and processing), numbering, totals and tax, PDF rendering, public links and P1/P2/P5, authorization capture (online and in-shop), authorization PDF, recommendations, estimate expiry job, email delivery through Postmark, follow-up scheduling infrastructure (sending of follow-ups lands in M5).
- **Acceptance:** E2E #3 and #11; sent versions cannot be modified (constraint tests); authorization evidence contains every Section 16 field; production refuses to send when the Texas authorization policy is not approved.
- **Dependencies:** M2.
- **Risks:** signature capture across iOS Safari and Android Chrome; PDF fidelity.
- **Tests:** totals math, state machines, immutability triggers, public-link security, data-exposure allowlist.

M4. Invoice to card payment (≈ 2 weeks)

- **Delivers:** invoices from authorized estimates, draft edits, send, void and reissue, PDF, P3/P4, Stripe onboarding S25, Checkout Sessions, Stripe webhooks and mapping, cash and check, refunds and disputes mirrored, receipts, nightly reconciliation.
- **Acceptance:** E2E #4 and #5; duplicate and out-of-order Stripe events produce one payment; delayed webhook path resolves through the return-page retrieval; refund in the Stripe dashboard returns the invoice to balance due.
- **Dependencies:** M3; Stripe platform configuration confirmed.
- **Risks:** connected-account configuration under Stripe-managed pricing (Open Question 3).
- **Tests:** webhook suite, overpayment race, reconciliation mismatch alert.

M5. Business texting (≈ 2.5 weeks)

- **Delivers:** texting registration flow S26 and admin I3, Twilio ISV adapter (subaccount, profile, brand, campaign, number), status sync, inbound and outbound SMS, inbox S15/S16, delivery callbacks, consent ledger and state, STOP/START/HELP, channel selection for estimate and invoice sends, email fallback, the two reminders with send windows, templates S27.

- **Acceptance:** E2E #6, #7, #8; a staging shop goes from `not_started` to `approved`; a rejection displays the reason and resubmits; STOP blocks sends at the database trigger even if the worker is bypassed in a test.
- **Dependencies:** M1 (can start in parallel with M2–M4); M3/M4 for sending documents by SMS.
- **Risks:** campaign review time and rejections (start our own and the staging campaign in week 1).
- **Tests:** keyword handling, status rank ordering, eligibility checks, send-window math across DST.

M6. Concierge import (≈ 1.5 weeks)

- **Delivers:** import pipeline, three source profiles, admin I2 with dry-run report and error CSV, historical sales, last-visit computation, attestation capture and consent events, reversal.
- **Acceptance:** E2E #12; re-running the same files creates nothing; a real (anonymized) Square and QuickBooks export from Phase 0 shops imports with < 2% error rows.
- **Dependencies:** M2.
- **Risks:** export format drift; dirty phone data.
- **Tests:** fixture files per profile, idempotency, partial failure and resume, reversal of touched vs untouched records.

M7. Money Finder and outreach (≈ 2 weeks)

- **Delivers:** the five queries, S1–S4, dismiss and snooze, composer with preview, eligibility per Section 14, bulk safeguards and first-batch review, batch sending, tracking links `/r`, interactions (reply, click, link approval, link payment, owner-marked).
- **Acceptance:** fixture tests for every list rule; an imported shop sees correct counts and values; ineligible recipients are shown and skipped with reasons; cooldown and daily caps enforced; auto-pause triggers on a simulated opt-out spike.
- **Dependencies:** M3, M5, M6 (M4 for unpaid invoices).
- **Risks:** consent rules leave many lapsed customers email-only (business, not technical, risk; surfaced in UI).
- **Tests:** list fixtures, eligibility matrix, rate limits, template validation.

M8. Recovered revenue attribution (≈ 1.5 weeks)

- **Delivers:** attribution rules engine, evaluate and sweep jobs, `source_outreach_id` propagation from threads and items, S5/S6, owner reversal, admin manual attribution.
- **Acceptance:** the four Section 12 examples reproduce exactly, including explanation text; refunds and voids adjust or reverse; no screen sums direct and influenced.
- **Dependencies:** M4, M7.
- **Risks:** owners forgetting to create work from the thread (mitigated by Re-quote and Book buttons on the outreach row and the "Customer responded" action).
- **Tests:** rule table tests, conflict scenarios, determinism of the sweep, E2E #9.

M9. Hardening and design-partner release (≈ 2 weeks)

- **Delivers:** all alerts in Section 20, failure drills for every Section 19 row, internal security review and fixes (independent penetration test moves to commercial V1), performance sanity at 20,000 customers, approved Texas legal policies loaded in production, support runbooks, data export tool, onboarding of shop #1.
- **Acceptance:** Definition of Done (Section 27) fully checked.
- **Dependencies:** M1–M8; counsel sign-off.
- **Risks:** legal review timing gates the first real send, not the build.
- **Tests:** full blocking E2E suite on staging and production smoke with a team-owned test shop.

27. Definition of done for the first Texas shop

Shop #1 goes live only when every box is checked and signed off by the engineering lead and the founder.

Security and tenant isolation

- [ ] RLS, cross-tenant and public-link suites green on the release commit; production RLS canary running
- [ ] Security review completed: dependency and secret scanning clean; manual review of auth, sessions, public links, webhooks, file access and admin by an engineer who did not build them; all findings fixed. Independent penetration test scheduled before commercial V1 (run it now only if quick and inexpensive)
- [ ] Owner MFA on new devices; admin SSO with MFA; view-as read-only and logged

- [ ] Secrets only in environment settings; EIN and subaccount tokens encrypted; access list reviewed

Backups and recovery

- [ ] PITR enabled; off-site nightly backup running; one full restore drill documented with time to restore
- [ ] Runbook: restore, provider outage, webhook replay, recovery kill switch

Monitoring

- [ ] Every Section 20 alert firing correctly in a staged test; on-call phone set
- [ ] Uptime checks on app, public pages and webhook routes

Payments

- [ ] Live Stripe platform approved; shop #1 `charges_enabled` and `payouts_enabled`
- [ ] Live $1 test payment by the team on shop #1's account, refunded in the dashboard, mirrored correctly
- [ ] Reconciliation job clean for 3 consecutive nights on staging

Messaging

- [ ] Shop #1 texting state `approved`, number active, test text both ways
- [ ] STOP, START, HELP verified on the live number
- [ ] Send windows, cooldowns, daily cap and auto-pause verified
- [ ] Email domain authenticated (SPF, DKIM, DMARC); bounce webhook verified

Legal configuration [COUNSEL]

- [ ] Texas policies approved and loaded: estimate authorization, invoice terms, SMS consent disclosure, recovery footer, retention rules, texting attestation
- [ ] Written counsel answers to Open Questions 1, 2 and 4 reflected in policy flags
- [ ] Terms of service and privacy policy for shops published; data processing terms signed by shop #1

Import verification

- [ ] Dry run reviewed with the owner on a call; attestation recorded
- [ ] Spot check of 25 random customers against the source export (name, phone, last visit, totals)
- [ ] Money Finder counts reviewed with the owner and make sense to him

Customer-facing documents

- [ ] Estimate, authorization and invoice PDFs reviewed on paper and phone; shop details correct
- [ ] Public pages tested on a real iPhone (Safari) and Android (Chrome), including signature
- [ ] Tax rate and line-type tax defaults confirmed with the owner

Provider failure behavior

- [ ] Each Section 19 row exercised on staging with the expected owner-visible result

Support tooling

- [ ] Admin console: tenants, view-as, imports, registrations, webhooks, policies, corrections, export
- [ ] Support can find any request by ID and any message or payment by provider ID
- [ ] Owner-facing help: a one-page "how to" for estimates, payments, Money Finder; support phone number

Data export

- [ ] Full export (CSV per entity plus files) generated for a test shop and verified to open in a spreadsheet

28. Open questions

Only questions whose answers change what gets built or when the first real message can go out. Everything else in this spec is decided.

#	Question	Why it matters	Recommended answer	What changes
1	What consent basis lets a shop text its existing and imported customers about declined work, open estimates, unpaid invoices, lapsed visits and service due?	Determines whether the Money Finder can use SMS for most of an imported book. The core product promise depends on it	Engage Texas counsel in week 1. Ask category by category: unpaid invoices, open estimates and declined work the customer requested may be informational; lapsed and service-due outreach is more likely marketing. Build the conservative default now	The Texas policy carries an allowlist of recovery categories permitted on transactional consent and a flag for import attestation. No code change, only policy values. If everything requires express written consent, lapsed outreach becomes email-first and the opt-in checkbox on estimates becomes a key growth lever
2	Which Twilio campaign use case covers recovery outreach?	Wrong choice risks rejection or filtering of every shop's messages	Register one Mixed campaign per shop with samples for transactional, reminder and recovery messages, confirmed with Twilio before M5	If a separate marketing campaign is required, the registration flow registers two campaigns per shop and the sender picks the Messaging Service by purpose

#	Question	Why it matters	Recommended answer	What changes
3	Exact Stripe connected-account configuration under Stripe-managed pricing	Determines onboarding UX, who handles refunds and disputes, and whether the shop has its own dashboard	Accounts where the shop pays Stripe fees, holds its own full dashboard and handles refunds and disputes there (Standard-type), with direct charges	If Stripe requires Express-type accounts, we must build a minimal refund action and dispute notice in-app, which is otherwise NOT NOW
4	Texas authorization text, estimate requirements, record retention and messaging hours	Required copy and retention for documents and messages	Counsel drafts all six policies; we load them as approved policy versions	Policy content only. Gates the first real send, not the build
5	Must shops invoice work that had no estimate?	Section 6 builds invoices only from approved estimates; small walk-in jobs may feel slow	Keep Section 6. Make the quick path fast: one-line estimate → "Customer signs here" → invoice, under a minute. Review after four weeks of design-partner use	If allowed, invoices gain an `authorization_id` - optional path with an in-person verbal authorization record, which itself needs counsel review
6	Which line types are taxable by default in Texas?	Wrong defaults overcharge or undercharge customers	Per-line taxable flag with defaults set per business during onboarding; confirm the defaults with a Texas CPA before shop #1	Default values only

#	Question	Why it matters	Recommended answer	What changes
7	Product name and short-link domain	Every SMS and public page carries it; needed before M3 links go out	Choose the name and buy a short domain before M3	Configuration only

29. Not now list

A coding agent must not build any of these during the Design Partner MVP, even when the schema could support them. If a task seems to need one, stop and ask.

Area	NOT NOW
Customers	Fleet accounts; multiple contacts per customer; households; customer addresses; custom fields; tags
Vehicles	Mileage-based service prediction; CARFAX or plate lookup; vehicle photos
Work	Jobs board; repair orders as a separate object; mobile-mechanic-specific workflow; routes, dispatch, travel windows
Schedule	Drag-and-drop calendar; week/month views; tech assignment; capacity planning; online booking; appointment reminder texts
Estimates	Advanced service catalog beyond 25 saved lines; packages; complex discounts; deposits; pricing matrix; labor guide integrations; parts ordering; digital vehicle inspections
Invoices and payments	Invoices without an estimate (pending Open Question 5); partial-payment UI; payment allocations; ACH; card on file; refund UI; Tap to Pay; payment plans; consumer financing; statements

Area	NOT NOW
Messaging	Missed-call text-back; voice; IVR; call recording; AI receptionist; MMS sending; Facebook, Instagram or Google Business messaging; marketing campaigns; review automation
Automation	Any automation beyond invoice reminder and estimate follow-up; automation builder; custom timing; automatic recovery sending
AI	AI-written messages; note rewriting; business-data chatbot; AI estimates
Accounts	Roles beyond Owner/Staff; role configuration UI; more than two users; SSO for shops; multi-location UI
Business	Self-service imports and column mapping; self-service SaaS billing, plans and entitlements; QuickBooks live sync; self-serve data export button; reports beyond recovered revenue
Platform	Native mobile apps; offline mode; push notifications; public API; integrations marketplace; other trades and vertical modules; states other than Texas

30. Final build order

Thirteen steps. Each one produces something a person can use on staging, and nothing starts before what it depends on is proven.

#	Step	What becomes usable	Must already exist	Proof it works
1	Tenant foundation	Two isolated test shops in every environment; deploy and migrate pipeline; backups	Nothing	RLS and isolation suites green; CI rejects a table without RLS; restore drill done

#	Step	What becomes usable	Must already exist	Proof it works
2	Sign-in, team and admin	Owner and Staff log in; Owner invites Staff; admin view-as	Step 1	Auth E2E; view-as logged and read-only
3	Customer and vehicle memory	Find any customer by phone, plate or VIN; vehicles, mileage, notes, timeline, Today list	Step 2	E2E #2; search fixtures
4	Estimate drafts	Build estimates with lines, photos, tax, saved lines	Step 3	Totals and photo-processing tests
5	Send and authorize	Estimate emailed; customer approves some lines, declines others, signs; declined work recorded	Step 4, legal policy records (draft in staging)	E2E #3 and #11; evidence complete
6	Invoices	Invoice from authorized estimate; send; void and reissue; cash and check	Step 5	E2E #5; immutability tests
7	Card payments	Shop connects Stripe; customer pays by card; webhooks reconcile	Step 6; Stripe platform ready	E2E #4; duplicate and out-of-order webhook tests
8	Texting registration	Shop submits registration and sees honest status; number provisioned on approval	Steps 1–2; our Twilio ISV setup	Staging shop reaches `approved`; rejection path works

#	Step	What becomes usable	Must already exist	Proof it works
9	Inbox, consent and reminders	Two-way texting; STOP; documents sent by SMS; email fallback; invoice reminder and estimate follow-up	Steps 5–8	E2E #6, #7, #8
10	Concierge import	Real exports loaded with dry run, report and reversal	Step 3	E2E #12; Phase 0 exports import cleanly
11	Money Finder and outreach	Five lists with dollars; owner selects, edits, sends; replies and clicks tracked	Steps 5–7, 9, 10	List fixtures; eligibility and safeguards tests
12	Recovered revenue	Direct and influenced records with explanations; report; reversal	Steps 7, 11	Section 12 examples reproduce exactly; E2E #9
13	Harden and launch shop #1	Production-ready system with alerts, drills, security review and approved Texas policies	Steps 1–12; counsel answers	Every Section 27 box checked